Drones que buscan personas para principiantes

Percepción, estimación y control de un Tarot 650 que busca personas perdidas en el bosque por sí solo

ROS 2 Jazzy · Gazebo Harmonic · PX4 · ArduPilot · YOLO26

Dr. Jairo Olguín Roque
FES Acatlán, UNAM

Octubre de 2026

Índice general

Cómo usar este libro vii

I Antes de despegar: lo básico 1

1. El proyecto de un vistazo 2

1.1. El problema . . . 2
1.2. Qué hace la simulación . . . 2
1.3. Arquitectura . . . 3
1.4. Un vuelo completo . . . 4
1.5. Convenciones de marcos . . . 4

2. ¿Qué es un dron y cómo vuela? 6

2.1. Drones, multirrotores y cuadrirrotores . . . 6
2.2. Las partes . . . 7
2.3. ¿Por qué vuela? El empuje . . . 7
2.4. Cómo se mueve: subir, alabeo, cabeceo y guiñada . . . 8
2.5. Configuración "+" y "X" . . . 10
2.6. El Tarot 650 . . . 10
2.7. Para qué sirve un dron en una búsqueda . . . 11

3. Física sin miedo 12

3.1. Vectores . . . 12
3.2. Las leyes de Newton aplicadas al dron . . . 12
3.3. Empuje: el cuadrado de la velocidad . . . 12
3.4. Inercia de rotación y par . . . 13
3.5. Marcos de referencia: ¿desde dónde mido? . . . 13
3.6. Rotaciones . . . 14

3.7. Las ecuaciones del dron en una página . . . 15

4. ¿Qué es una simulación? Gazebo y SITL 17

4.1. ¿Por qué simular? . . . 17
4.2. Gazebo . . . 17
4.3. SDF: cómo se describe un robot . . . 18
4.4. Personas con etiqueta . . . 18
4.5. Tiempo simulado . . . 18
4.6. SITL: el piloto real en la computadora . . . 19

5. ROS 2 y PX4 para principiantes 20

5.1. La terminal en cinco minutos . . . 20
5.2. ROS 2 . . . 20
5.3. PX4 . . . 21
5.4. El puente: uXRCE-DDS . . . 21
5.5. Workspace y compilación . . . 22

II Ver y ubicar: percepción y estimación 23

6. Qué es una imagen para una computadora 24

6.1. Una imagen es una tabla de números . . . 24
6.2. Cómo forma una imagen una cámara . . . 25
6.3. Una cámara que mira hacia abajo . . . 25
6.4. Del píxel al mundo . . . 26

7. Redes neuronales y YOLO sin complicaciones 27

7.1. Programar contra aprender . . . 27
7.2. La neurona artificial . . . 27
7.3. Convoluciones . . . 27
7.4. YOLO . . . 28
7.5. Cómo se mide si funciona . . . 29

8. El detector de personas 30

8.1. Etiquetas automáticas . . . 30
8.2. Entrenamiento . . . 31

8.3. Por estrato: dónde falla . . . 33
8.4. El nodo detector . . . 33

9. Ubicar a la persona: localización y fusión 34
9.1. Del recuadro al suelo: el rayo . . . 34
9.2. ¿Qué tan grande es el error? . . . 35
9.3. Juntar detecciones: asociación y fusión . . . 36

III El control: de la cascada a la red neuronal 38

10. Control automático desde cero 39
10.1. ¿Qué es controlar? . . . 39
10.2. Lazo abierto y lazo cerrado . . . 39
10.3. El controlador proporcional (P) . . . 40
10.4. El término derivativo (D): frenar a tiempo . . . 40
10.5. El término integral (I): no rendirse . . . 40
10.6. Cómo se lee una respuesta . . . 41
10.7. Control en cascada: así se controla un dron . . . 42

11. Una red neuronal que vuela el dron 43
11.1. La idea . . . 43
11.2. Aprender imitando: clonación de comportamiento y DAgger . . . 44
11.3. Aprender por refuerzo: PPO . . . 45
11.4. De Python a PX4 . . . 45
11.5. Tres problemas de la primera versión y su solución . . . 46
11.5.1. Un sesgo de 20 cm . . . 46
11.5.2. La red iba a la mitad de velocidad . . . 46
11.5.3. PPO divergía . . . 47
11.6. Resultado en la misión . . . 48

IV Construir la simulación 50

12. Instalación paso a paso 51
12.1. 0. ROS 2 Jazzy y Gazebo Harmonic . . . 51
12.2. 1. PX4 release/1.17 . . . 51

12.3. 2. Agente Micro XRCE-DDS 2.4.3 . . . 51
12.4. 3. Workspace . . . 52
12.5. 4. Entornos de Python . . . 52
12.6. 5. Vehículo, red y compilación de PX4 . . . 52
12.7. 6. ArduPilot (opcional) . . . 53
12.8. Comprobación . . . 53

13. Dónde está cada cosa **54**
13.1. Nodos y módulos . . . 54
13.2. Programas de referencia (`codigo/`) . . . 55
13.3. Scripts . . . 55

14. El Tarot 650 en Gazebo **56**
14.1. El tarot650_busqueda . . . 56
14.2. Supuestos que hay que reemplazar . . . 57
14.3. La variante para ArduPilot . . . 58
14.4. El x500 de PX4 sigue disponible . . . 58
14.5. Las personas . . . 60

15. Un bosque hecho por factores **61**
15.1. Por qué un bosque "por factores" . . . 61
15.2. Cobertura de copas . . . 61
15.3. Las personas . . . 62
15.4. Georreferencia . . . 63

V La misión: buscar sin piloto **64**

16. Conceptos de autonomía **65**
16.1. Percibir, decidir, actuar . . . 65
16.2. Máquinas de estados . . . 65
16.3. Cubrir toda la zona . . . 65
16.4. Volver a mirar: el valor de la información . . . 66
16.5. GPS: de metros a latitud y longitud . . . 66
16.6. LiDAR . . . 67

17. La misión de búsqueda, pieza por pieza **69**

17.1. El ciclo . . . 69
17.2. Con la red neuronal . . . 69
17.3. Avisos con GPS . . . 70
17.4. Evasión de obstáculos . . . 71
17.5. Métricas . . . 72

18. Volar con PX4 o con ArduPilot **73**

18.1. Con PX4 (`piloto:=px4`, por omisión) . . . 73
18.2. Con ArduPilot (`piloto:=ardupilot`) . . . 73
18.2.1. ArduPilot vuela con su EKF3 . . . 74
18.3. La estación de tierra: MAVProxy . . . 74
18.4. Una simulación a la vez . . . 75

VI Experimentos y manos a la obra **76**

19. La campaña experimental **77**

19.1. Las preguntas . . . 77
19.2. El diseño . . . 77
19.3. Wilcoxon y Holm . . . 78
19.4. Resultados . . . 78

20. Tu primer vuelo, paso a paso **80**

20.1. Antes de empezar . . . 80
20.2. Paso 1: lanzar . . . 80
20.3. Paso 2: mirar . . . 81
20.4. Paso 3: leer el resultado . . . 81
20.5. Paso 4: cambiar una cosa . . . 81
20.6. Si algo falla . . . 82

21. Referencia rápida **83**

21.1. Argumentos de `simulacion.launch.py` . . . 83
21.2. Parámetros de `mision_node` . . . 84
21.3. Tópicos . . . 84
21.4. Programas . . . 84

22. Lo que se verificó y lo que falta **86**

22.1. Diferencias con el diseño original 86

22.2. Lo que se comprobó 87

22.3. Pendientes 87

A. Glosario **88**

Cómo usar este libro

¿Para quién es?

Para cualquier persona que quiera entender, desde cero, cómo funciona un dron que busca personas por sí solo: estudiantes, ingenieros de otras áreas o curiosos. No necesitas saber de drones, de visión computacional ni de redes neuronales. Ayuda saber usar una computadora con Linux y recordar algo de física y matemáticas de bachillerato; cuando haga falta algo más, se explica en el momento.

El libro cuenta una historia concreta: una simulación en la que un dron Tarot 650 despega, recorre en franjas un bosque de 120×120 metros, mira hacia abajo con su cámara, reconoce a las personas con una red neuronal (YOLO), calcula en qué punto del suelo está cada una, avisa sus coordenadas GPS y aterriza. El vuelo lo controla un piloto automático real (PX4 o ArduPilot) o una red neuronal entrenada para imitarlo. Cada pieza de esa historia es una excusa para explicar un concepto.

Cómo está organizado

Parte I. Lo básico. Qué es un dron y cómo vuela, la física mínima, qué es una simulación, y ROS 2 y PX4, que conectan todas las piezas.

Parte II. Ver y ubicar. Qué es una imagen, cómo aprende una red neuronal a reconocer personas y cómo se pasa de un recuadro en la imagen a un punto en el mapa, con su incertidumbre.

Parte III. El control. Cómo se mantiene el dron en su ruta: control en cascada y una red neuronal que aprende a volar imitando al piloto automático.

Parte IV. Construir la simulación. La instalación y cómo se metieron en Gazebo el Tarot 650, su cámara, el bosque y las personas.

Parte V. La misión. Cómo el dron decide qué hacer: el barrido, la re-observación de casos dudosos, los avisos GPS y la evasión de obstáculos, con PX4 o con ArduPilot.

Parte VI. Experimentos y manos a la obra. Cómo se diseña y analiza una campaña de cientos de vuelos, un tutorial para tu primer vuelo y la referencia de órdenes.

Al final hay un **glosario**. Si solo quieres usar la simulación, ve directo al capítulo 20.

Los recuadros

Recuerda

Lo más importante del tema, en pocas palabras.

Consejo

Trucos prácticos que ahorran tiempo o problemas.

Ejemplo de la vida diaria

Una comparación con algo cotidiano.

Para curiosos (te lo puedes saltar)

Detalles matemáticos o de programación que puedes saltarte la primera vez.

Cuidado

Errores fáciles de cometer.

Problema encontrado y cómo se resolvió

Problemas reales que aparecieron durante el desarrollo y cómo se resolvieron.

Pruébalo tú

Algo que puedes hacer en la simulación para comprobar lo que acabas de leer.

Convenciones

- Las órdenes de terminal van en recuadros grises:

```
ros2 launch mision simulacion.launch.py
```

- Nombres de archivos, programas y tópicos en letra de máquina: `localizador_node`.
- `~` es tu carpeta personal. El paquete principal está en `~/ros2_ws/src/mision`.
- Distancias en metros, tiempos en segundos, ángulos en radianes o grados (se indica).

Parte I

Antes de despegar: lo básico

Capítulo 1

El proyecto de un vistazo

1.1. El problema

Cuando una persona se pierde en un bosque, encontrarla pronto puede salvarle la vida. Un dron con cámara recorre en minutos lo que a una brigada a pie le tomaría horas, y lo hace sin arriesgar a nadie. Pero desde el aire una persona ocupa unos pocos píxeles, las copas de los árboles la tapan, la luz cambia y el viento mueve al dron. Para que el dron sirva hay que resolver tres preguntas:

1. **¿Hay alguien en la imagen?** (percepción: detector de personas);
2. **¿Dónde está en el mapa, y qué tan seguro estoy?** (estimación: localización en el terreno y fusión de observaciones);
3. **¿Cómo recorro el bosque sin perder a nadie?** (control y misión).

1.2. Qué hace la simulación

- **Bosque por factores:** se genera un bosque con tres niveles de cobertura de copas (15, 45 y 75 %), dos condiciones de luz (sol alto o sombras largas), con o sin viento, y de una a tres personas de pie, sentadas o acostadas en lugares al azar.
- **Dron:** un Tarot 650 de 2.25 kg (modelo hecho con las mallas de su CAD) con una cámara de 1280×960 que mira hacia abajo, una IMU, GPS y un LiDAR. El x500 de PX4, con el que se hicieron las primeras corridas, sigue disponible (`vehiculo:=x500`).
- **Piloto:** PX4 1.17 o ArduCopter 4.7, en SITL (el mismo programa que volaría en el dron real).
- **Controlador:** la cascada del piloto automático o una red neuronal 15-64-32-4 que imita a la cascada y corre dentro de PX4 (módulo *Neural Control*).
- **Detector:** YOLO26s ajustado con 3000 imágenes del simulador (AP50 = 0.954).
- **Localizador:** corta el rayo de cada detección con el suelo, calcula su elipse de error y fusiona las detecciones de una misma persona.
- **Misión:** barrido en franjas a 30 m, re-observación de candidatas dudosas, avisos GPS, aterrizaje y métricas (personas localizadas, falsas alarmas, error).

1.3. Arquitectura

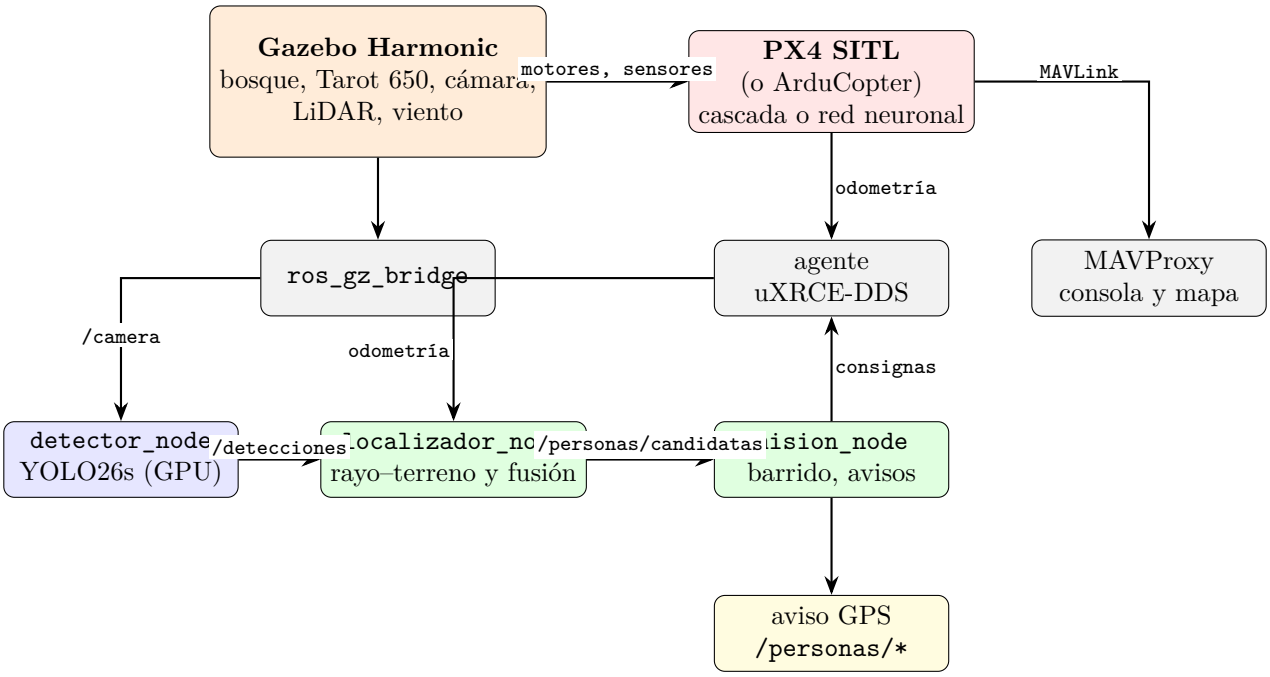


Figura 1.1: Programas y tópicos principales. La cadena es *imagen* → *detección* → *localización* → *fusión* → *decisión* → *consigna* → *control*.

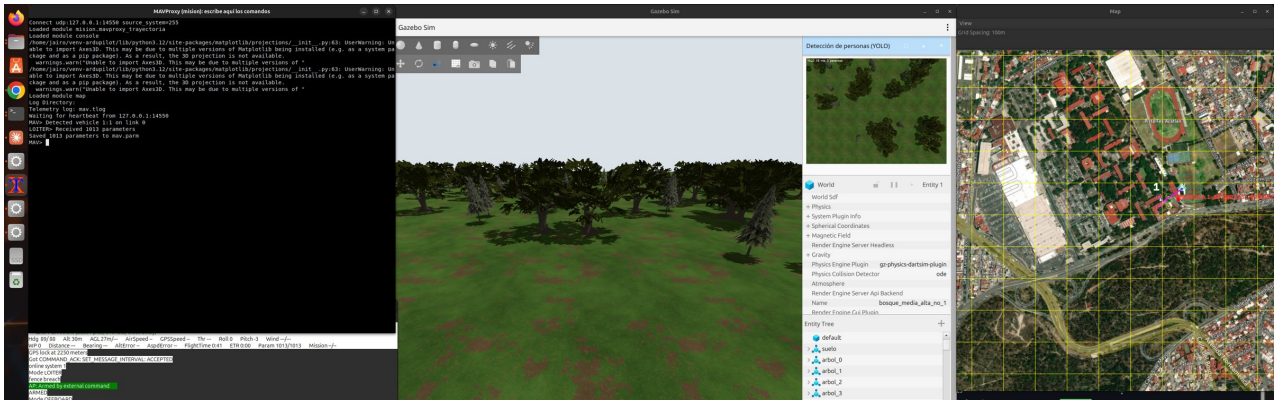


Figura 1.2: La simulación en marcha: a la izquierda la terminal y la consola de MAVProxy (la estación de tierra), al centro Gazebo con el panel de detección, a la derecha el mapa con la persona 1 ya localizada.

1.4. Un vuelo completo

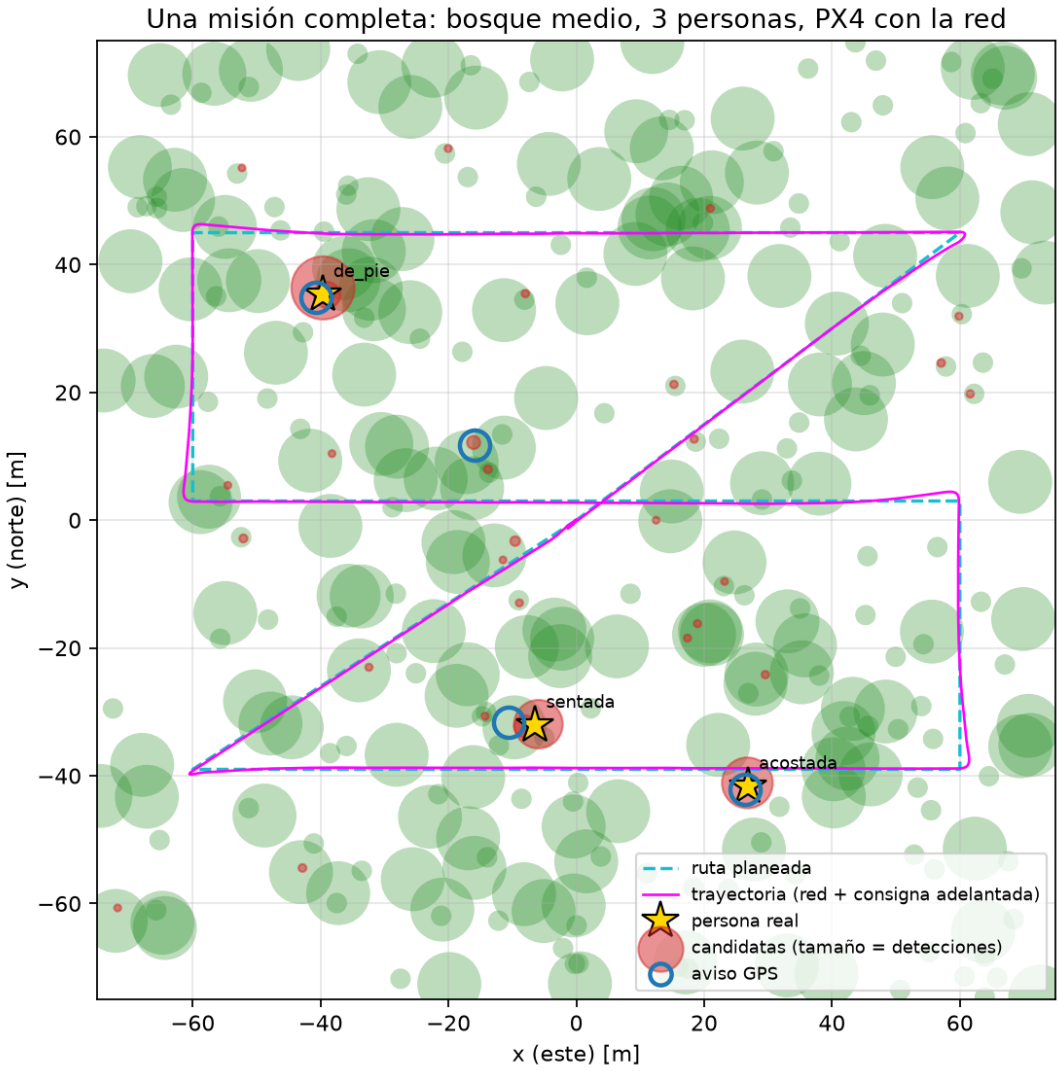


Figura 1.3: Una misión real de la simulación, vista desde arriba: el bosque (círculos verdes), la ruta planeada, lo que voló el dron, las candidatas que fue juntando el localizador, las tres personas reales y los avisos GPS.

La figura 1.3 resume todo el libro: el dron sigue franjas paralelas, el detector ve personas y también cosas que no lo son (puntos rojos), el localizador junta las detecciones de cada lugar, y la misión avisa cuando una candidata ya está confirmada. Las tres personas se encontraron con menos de 1 m de error; también hubo un aviso falso entre unos robles. Entender por qué pasa cada cosa es el objetivo de las páginas que siguen.

1.5. Convenciones de marcos

- **Mundo:** ENU (x este, y norte, z arriba), en metros, con origen en el centro del bosque. PX4 y ArduPilot usan NED: $N = y$, $E = x$, $D = -z$.

- **Cuerpo:** en ROS FLU (x al frente, y a la izquierda, z arriba); en PX4 FRD (y a la derecha, z abajo).
- El mundo está georreferenciado en 19.4826° N, 99.2447° O, 2250 m, así que el GPS del dron y los avisos dan coordenadas reales.

Capítulo 2

¿Qué es un dron y cómo vuela?

En este capítulo

Las partes de un cuadrirrotor, por qué vuela, cómo sube, avanza y gira con solo cambiar la velocidad de sus cuatro motores, y cómo es el Tarot 650 que usamos.

2.1. Drones, multirrotores y cuadrirrotores

"Dron" es el nombre popular de un **vehículo aéreo no tripulado** (UAV, por sus siglas en inglés): una aeronave sin piloto a bordo. Puede ser un avión de ala fija, un helicóptero o, el caso más común, un **multirrotor**: una aeronave con varias hélices horizontales. El de cuatro hélices se llama **cuadrirrotor** o **cuadricóptero**.

¿Por qué cuatro? Porque es el número mínimo con el que se puede controlar todo el movimiento sin piezas móviles más que los propios motores: no hay alerones, timones ni "plato cíclico" como en un helicóptero. Eso los hace baratos, robustos y fáciles de reparar; a cambio, son inestables por naturaleza: sin una computadora que corrija cientos de veces por segundo, se voltean.

Recuerda

Un cuadrirrotor no tiene superficies de control. Todo lo que hace (subir, inclinarse, girar) lo consigue cambiando la velocidad de sus cuatro motores. Por eso el *control* es el corazón del dron.

2.2. Las partes

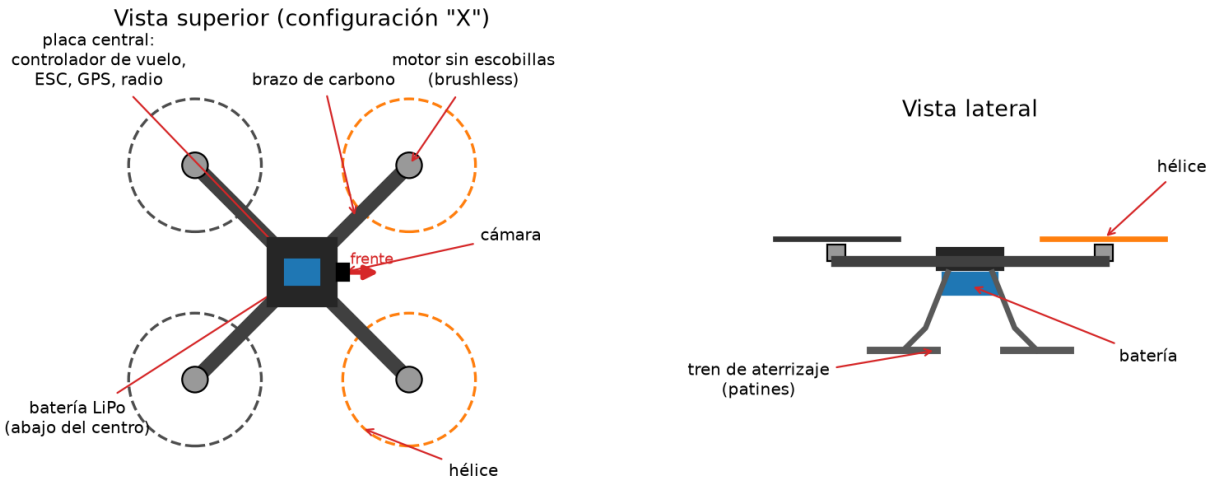


Figura 2.1: Partes de un cuadrirrotor.

Marco (frame). La estructura: una placa central y cuatro brazos. En el Tarot 650 son de fibra de carbono, ligera y rígida; en el modelo de la simulación hay unos 61 cm entre motores opuestos (el "650" del nombre es la medida comercial del marco, en milímetros).

Motores sin escobillas (brushless). Motores eléctricos trifásicos muy eficientes. Su constante KV dice cuántas revoluciones por minuto da por cada volt.

ESC (controlador electrónico de velocidad). Uno por motor. Recibe una orden ("gira a tanto") y genera las corrientes trifásicas para lograrlo.

Hélices. Convierten el giro en empuje. Se describen con dos números: diámetro y paso, en pulgadas. Una hélice de 13×4.5 tiene 13" de diámetro (33 cm) y 4.5" de paso (lo que avanzaría en una vuelta si se "atornillara" en el aire). Las del Tarot de la simulación son de 15" de diámetro (38 cm).

Batería LiPo. Polímero de litio: mucha energía para su peso. Se describen por el número de celdas en serie (4S, 6S: 3.7 V cada una) y su capacidad en mAh.

Controlador de vuelo. La computadora a bordo: lee los sensores y decide la velocidad de cada motor cientos de veces por segundo. Puede llevar ArduPilot o PX4. En nuestra simulación ese papel lo hacen PX4 o ArduPilot en SITL (y, en el modo neuronal, una red dentro de PX4).

Sensores. IMU (acelerómetros y giroscopios: cómo se mueve y gira), barómetro (altura), magnetómetro (brújula), GPS (posición) y, según la misión, cámaras y LiDAR.

Tren de aterrizaje. Patas o patines para posarse sin dañar la cámara ni la batería.

2.3. ¿Por qué vuela? El empuje

Una hélice que gira empuja aire hacia abajo; por la tercera ley de Newton (acción y reacción), el aire empuja a la hélice hacia arriba. A esa fuerza se le llama **empuje**. Crece con el *cuadrado* de la

velocidad de giro: al doble de velocidad, cuatro veces más empuje.

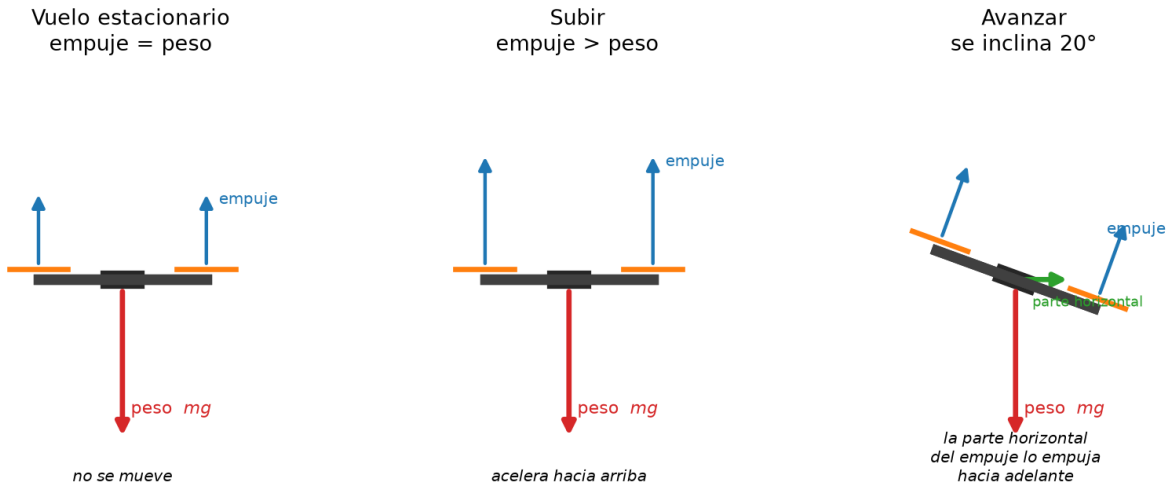


Figura 2.2: Las dos fuerzas principales: el empuje de las hélices (azul) y el peso (rojo).

- Si el empuje total es igual al peso, el dron se queda quieto en el aire: **vuelo estacionario** (*hover*).
- Si es mayor, sube; si es menor, baja.
- Para avanzar, el dron **se inclina**: el empuje ya no apunta hacia arriba, y su parte horizontal lo empuja hacia adelante (figura 2.2, derecha). Un cuadrirrotor no tiene "motor hacia adelante": avanza inclinándose, como una persona que se inclina para empezar a caminar.

Ejemplo de la vida diaria

Sostén un libro con la mano extendida, plano. Para moverlo hacia adelante sin empujarlo, inclínalo un poco: la fuerza de tu mano "hacia arriba" ahora tiene una parte hacia adelante. Así avanza un dron.

2.4. Cómo se mueve: subir, alabeo, cabeceo y guiñada

Un objeto en el espacio puede girar alrededor de tres ejes (figura 2.3). En aviación esos giros tienen nombre:

Alabeo (roll, ϕ): girar alrededor del eje que va al frente; el dron "se ladea".

Cabeceo (pitch, θ): girar alrededor del eje lateral; "asiente con la cabeza".

Guiñada (yaw, ψ): girar alrededor del eje vertical; cambia hacia dónde mira.

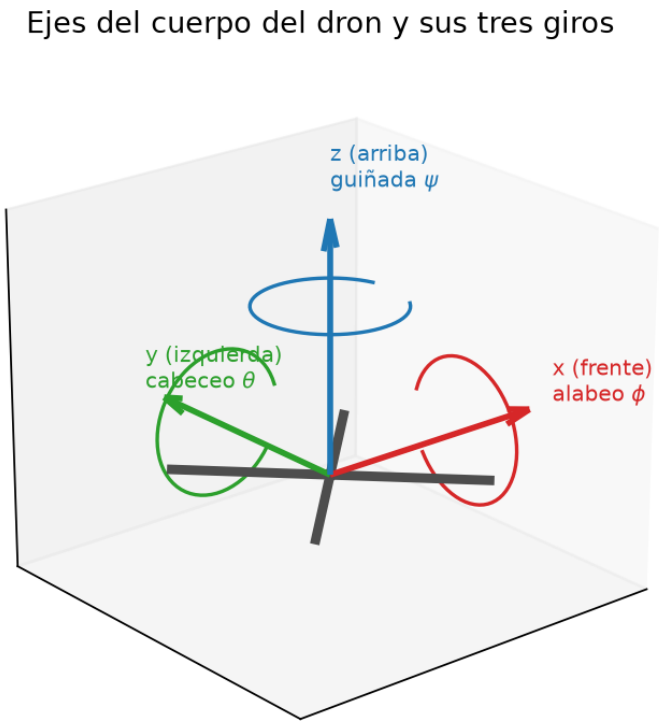


Figura 2.3: Ejes del cuerpo: x al frente, y a la izquierda, z arriba.

Con cuatro motores se controla todo así (figura 2.4):

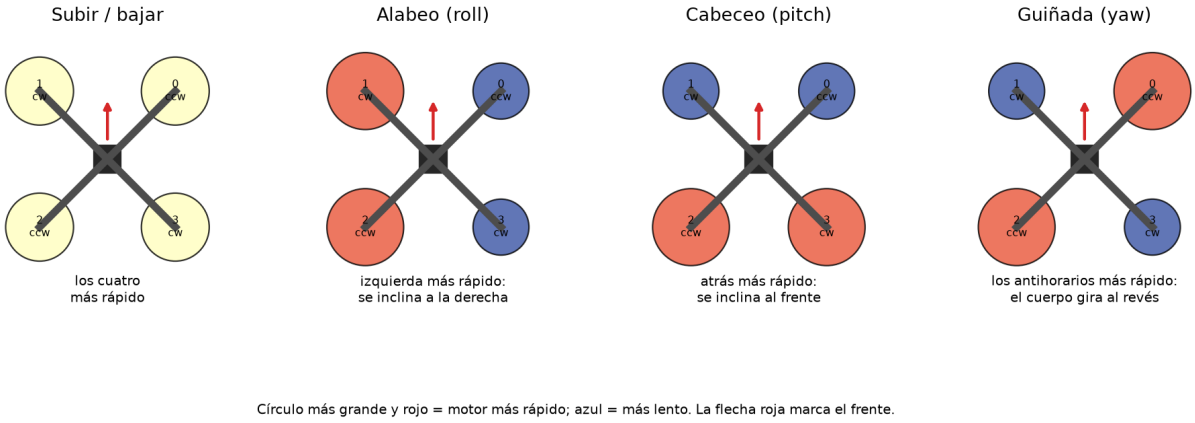


Figura 2.4: Cómo produce el dron cada movimiento solo cambiando la velocidad de los motores.

- **Subir o bajar:** los cuatro motores aceleran o frenan por igual.
- **Alabeo:** los motores de un lado giran más rápido que los del otro; el dron se inclina hacia el lado lento y empieza a moverse de lado.
- **Cabeceo:** igual, pero adelante contra atrás; así avanza o retrocede.
- **Guiñada:** aquí entra un truco. Cada hélice, al girar, "empuja" el aire de lado y el aire, por

reacción, intenta hacer girar al dron al revés. Por eso dos hélices giran en un sentido y dos en el otro: los efectos se cancelan. Si se aceleran las dos que giran en un sentido y se frenan las otras dos, el empuje total no cambia pero sobra par en un sentido, y el dron gira sobre sí mismo.

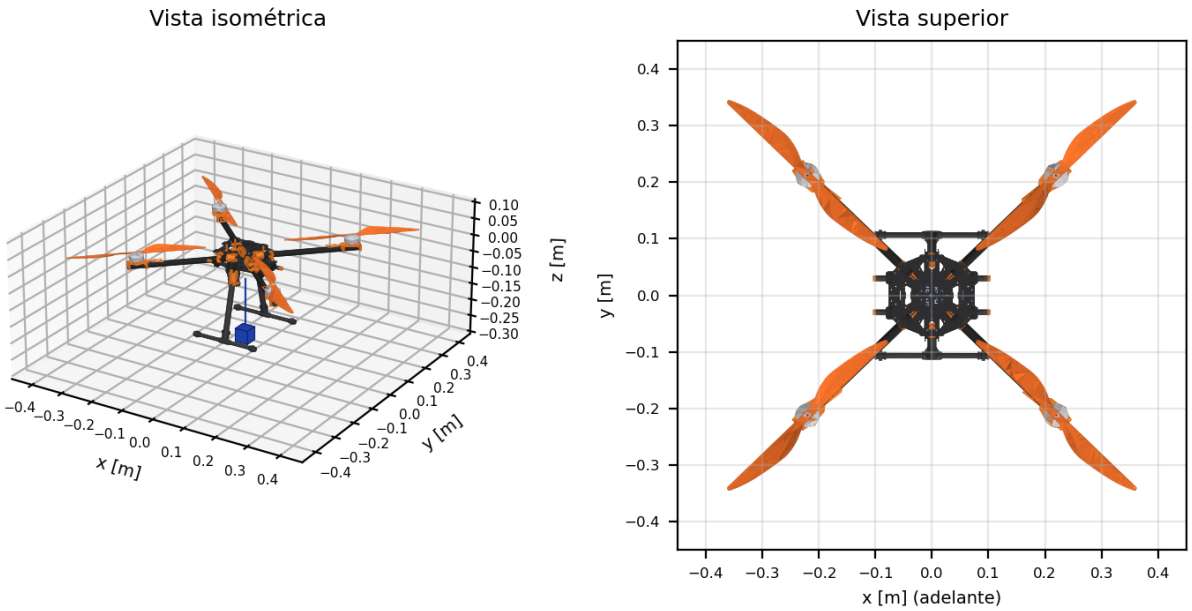
Recuerda

Cuatro motores, cuatro "perillas": empuje total, alabeo, cabeceo y guiñada. El avance y el movimiento lateral no se controlan directamente: se consiguen inclinando el dron. Por eso el control de un dron es *en cascada*: para ir a una posición hay que pedir una inclinación, y para lograr la inclinación hay que pedir velocidades a los motores.

2.5. Configuración "+" y "X"

Si un brazo apunta al frente, la configuración es "+"; si el frente queda entre dos brazos, es "X". La "X" es la más común: la cámara frontal no ve hélices y los cuatro motores trabajan al inclinarse hacia adelante. El Tarot 650 es "X".

2.6. El Tarot 650



Mallas del CAD del Tarot Ironman 650 (models/tarot650_busqueda). En azul, la cámara: 24 cm bajo el centro, mirando hacia abajo.

Figura 2.5: El Tarot 650 de la simulación, dibujado a partir de las mallas de su modelo de Gazebo (`herramientas/tarot650_figuras.py`).

El Tarot Ironman 650 es un marco de cuadrirrotor de fibra de carbono para cargar cámaras y otros sensores. El modelo de la simulación sale de las mallas de su CAD. El nuestro:

- motores a $(\pm 0.2198, \pm 0.212)$ m del centro, en "X";

- hélices de 15”;
- 2.25 kg en total (cuerpo de 2.12 kg, cuatro rotores de 20 g y una cámara de 50 g);
- cada motor da hasta 19.6 N; en total ≈3.6 veces su peso;
- una cámara de 1280 × 960 píxeles con campo horizontal de 1.74 rad (100°) que mira hacia abajo, 24 cm debajo del centro (bajo los patines).

Cuidado

La masa, las inercias, k_f y los retardos de los motores del modelo son *supuestos*, no medidos. Antes de confiar en la simulación para un dron real hay que reemplazarlos por los de ese dron (capítulo 14).

Para curiosos (te lo puedes saltar)

En vuelo estacionario cada motor sostiene un cuarto del peso: $2.25 \times 9.81/4 = 5.52$ N. Con $f = k_f\omega^2$ y $k_f = 4.24 \times 10^{-5}$, la velocidad necesaria es $\omega = \sqrt{5.52/k_f} = 361$ rad/s, unas 3450 revoluciones por minuto, un poco más de la mitad de su máximo de 680 rad/s.

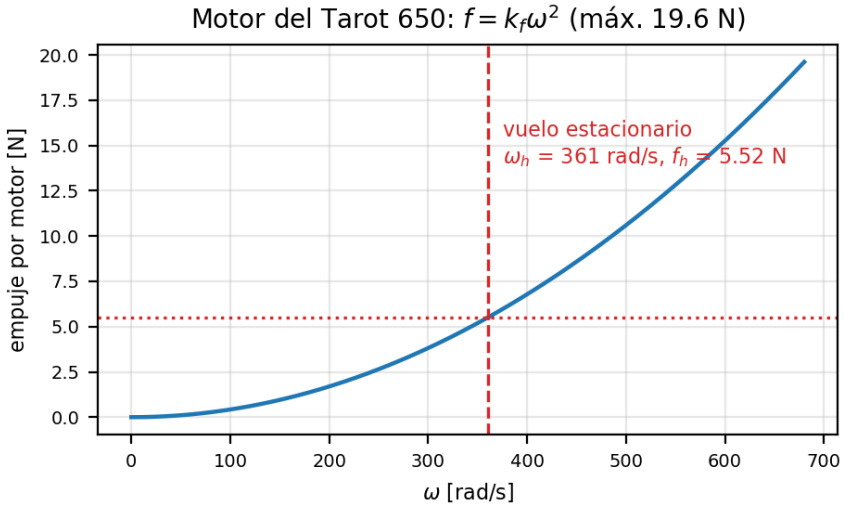


Figura 2.6: Empuje de un motor del Tarot 650 contra su velocidad de giro.

2.7. Para qué sirve un dron en una búsqueda

Un dron puede recorrer zonas de difícil acceso, ver desde arriba claros que a pie pasarían desapercibidos, dar las coordenadas exactas de lo que encuentra y repetir la búsqueda las veces que haga falta. Para hacerlo solo necesita tres capacidades, que son las partes centrales del libro:

1. **percepción y estimación**: reconocer personas y ubicarlas en el mapa (Parte II);
2. **control**: seguir su ruta con precisión aunque haya viento (Parte III);
3. **autonomía**: decidir a dónde ir y cuándo avisar (Parte V).

Capítulo 3

Física sin miedo

En este capítulo

Vectores, las leyes de Newton aplicadas al dron, por qué el empuje crece con el cuadrado de la velocidad, qué es la inercia, qué son los marcos de referencia (ENU, NED, cuerpo) y cómo se describe una rotación. Al final, las ecuaciones del dron en una página.

3.1. Vectores

Muchas cantidades físicas tienen tamaño y dirección: la velocidad del dron es "3 m/s *hacia el este*". Son **vectores** y se dibujan como flechas; en tres dimensiones tienen tres componentes, $v = (v_x, v_y, v_z)$. Posición, velocidad, aceleración, fuerza y velocidad angular son vectores.

3.2. Las leyes de Newton aplicadas al dron

1. **Inercia.** Un cuerpo sigue como está mientras nadie lo empuje: un dron que avanza y deja de inclinarse sigue avanzando un rato.
2. $F = ma$**.** Para el dron,
$$m\,\ddot{p} = \underbrace{T\,R\,e_3}_{\text{empuje}}\ \underbrace{-\,mg\,e_3}_{\text{peso}}\ +\ \underbrace{d(t)}_{\text{viento}} \tag{3.1}$$
con T el empuje total, Re_3 la dirección "arriba" del dron (cambia si se inclina) y $e_3 = (0, 0, 1)$.
3. **Acción y reacción.** La hélice empuja aire hacia abajo y el aire la empuja hacia arriba; además, al girarla, el motor intenta girar al dron al revés (por eso las hélices van en pares que giran en sentidos opuestos).

3.3. Empuje: el cuadrado de la velocidad

El empuje de una hélice es aproximadamente $f = k_f\,\omega^2$. En el Tarot 650, $k_f = 4.24\times10^{-5}$ N/(rad/s)^2 y $\omega_{max} = 680$ rad/s, así que cada motor da hasta 19.6 N. Cada hélice produce además un par de arrastre $k_m f$ ($k_m = 0.016$ m) que intenta girar al dron.

Recuerda

Los motores no cambian de velocidad al instante: en el Tarot simulado tienen constantes de tiempo de 0.03 s al acelerar y 0.05 s al frenar (en el x500, 12.5 y 25 ms). Ese retardo, aunque pequeño, limita qué tan agresivo puede ser el control.

3.4. Inercia de rotación y par

Para girar importa el **momento de inercia** J: cuánto se resiste un cuerpo a cambiar su giro. El Tarot 650 completo (con sus rotores) tiene 0.0341, 0.0361 y 0.0660 $\mathrm{kg\,m^2}$ en sus tres ejes. Lo que hace girar es un **par** τ, una fuerza aplicada a cierta distancia del centro:

$$J\,\dot{\omega} = \tau - \omega \times J\omega. \tag{3.2}$$

Problema encontrado y cómo se resolvió

Un detalle de menos de un milímetro tuvo efectos visibles. La cámara (50 g) tiene su centro de masa a 3 cm del eje y corre el del dron completo 0.7 mm en el x500, el vehículo de las primeras corridas (0.67 mm en el Tarot). Eso produce un par pequeño pero constante que el controlador neuronal original no sabía compensar: el dron se quedaba inclinado $\approx 1°$ y a ≈ 0.2 m del punto deseado (capítulo 11).

3.5. Marcos de referencia: ¿desde dónde mido?

Mundo ENU (Este, Norte, Arriba): fijo al suelo, con origen en el centro del bosque. Lo usan Gazebo y ROS.

Mundo NED (Norte, Este, Abajo): el mismo lugar con otros ejes. Lo usan PX4 y ArduPilot.

Cuerpo: pegado al dron. En ROS: FLU (frente, izquierda, arriba); en PX4: FRD (frente, derecha, abajo).

Cámara: eje óptico hacia abajo, “derecha” y “abajo” de la imagen.

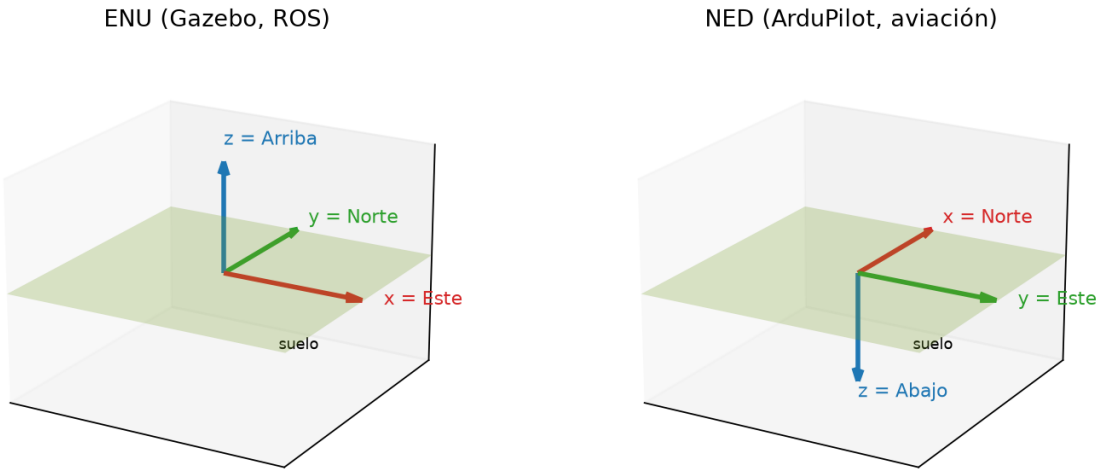


Figura 3.1: ENU y NED. Pasar de uno al otro es intercambiar x con y y cambiar el signo de z.

Cuidado

Confundir marcos es la causa de error número uno en drones. En este proyecto conviven cinco (ENU, NED, FLU, FRD y cámara). Cada conversión está en una función con nombre (`ned_a_enu`, `R_enu_flu`, `R_BC`...) y probada en `test/test_mision.py`.

3.6. Rotaciones

La orientación se describe con tres ángulos (alabeo ϕ, cabeceo θ, guiñada ψ) o con una **matriz de rotación** R, cuyas columnas son los ejes del dron vistos desde el mundo; PX4 la entrega como un **cuaternión** (cuatro números sin singularidades). En el localizador, $R = R_z(\psi)R_y(\theta)R_x(\phi)$.

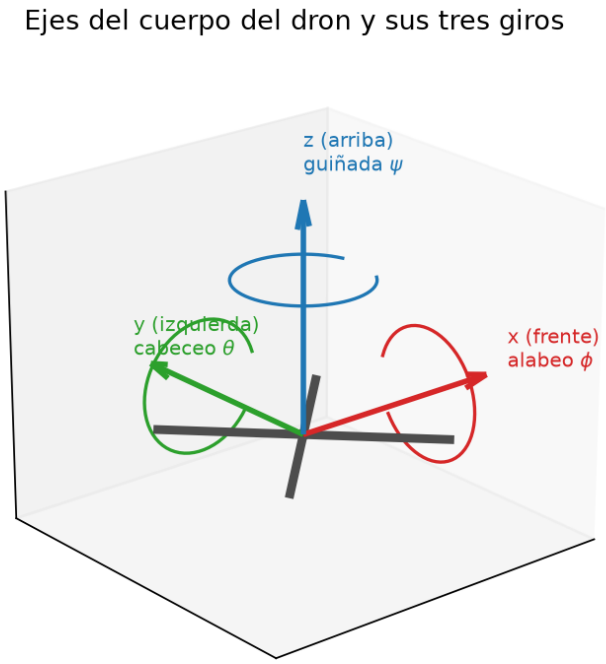


Figura 3.2: Ejes del cuerpo y sus tres giros.

3.7. Las ecuaciones del dron en una página

Modelo del cuadrirrotor

$$\begin{aligned}
\text{posición:} \quad & m\ddot{p} = TRe_3 - mg\,e_3 + d(t) \\
\text{orientación:} \quad & \dot{R} = R\hat{\omega} \\
\text{giro:} \quad & J\dot{\omega} = \tau - \omega \times J\omega \\
\text{motores:} \quad & f_i = k_f \omega_i^2, \quad \begin{bmatrix} T \\ \tau \end{bmatrix} = A\,f, \quad A = \begin{bmatrix} 1 & 1 & 1 & 1 \\ y_i & \cdots & & \\ -x_i & \cdots & & \\ -s_i k_m & \cdots & & \end{bmatrix}
\end{aligned}$$

`codigo/sim_modelo.py` simula este modelo con el control base y reproduce las cifras del diseño original del proyecto. Ese programa usa los parámetros del **x500** de PX4 (2.0 kg, $k_f = 8.54858\times10^{-6}$, $\omega_{max} = 1000$ rad/s), no los del Tarot: las cifras y la figura siguientes son del x500.

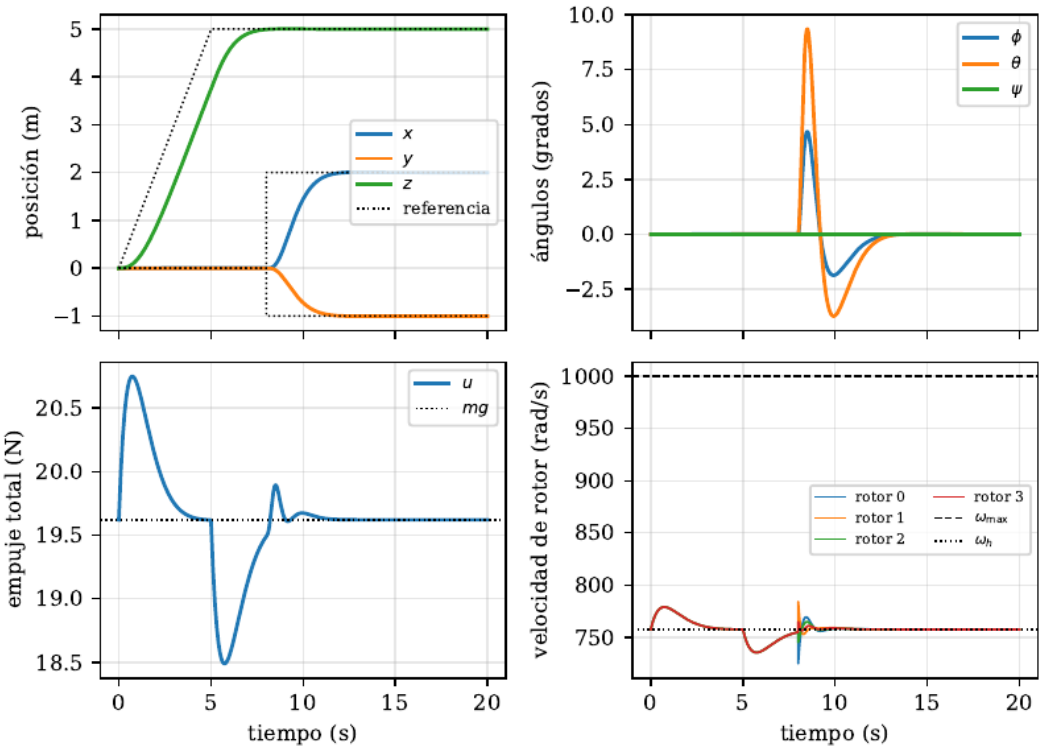


Figura 3.3: Salida de `sim_modelo.py`: respuesta del modelo del x500 con el control base.

Capítulo 4

¿Qué es una simulación? Gazebo y SITL

En este capítulo

Por qué se simula, qué hace Gazebo, cómo se describe un robot, qué son los plugins, el tiempo simulado y qué significa que PX4 corra "en el lazo" (SITL).

4.1. ¿Por qué simular?

Una búsqueda real necesita un dron, un bosque, voluntarios que hagan de personas perdidas y permisos; y un error de programación puede estrellar el dron. En simulación:

- los choques no cuestan nada;
- se repite exactamente el mismo bosque, con las mismas personas y el mismo viento, para comparar dos estrategias en igualdad de condiciones (imposible al aire libre);
- se conoce la verdad: dónde está cada persona, para medir el error de localización;
- se pueden correr cientos de vuelos sin supervisión (capítulo 19).

Recuerda

La simulación no reemplaza el vuelo real: lo prepara y permite responder preguntas que en campo costarían meses.

4.2. Gazebo

Gazebo Harmonic calcula en cada paso la física (motor DART), dibuja la escena (OGRE 2), genera las lecturas de los sensores (cámara, IMU, GPS, LiDAR) y ejecuta *plugins*: los motores, el viento (`WindEffects`), el piloto de ArduPilot o el selector de motores del proyecto.

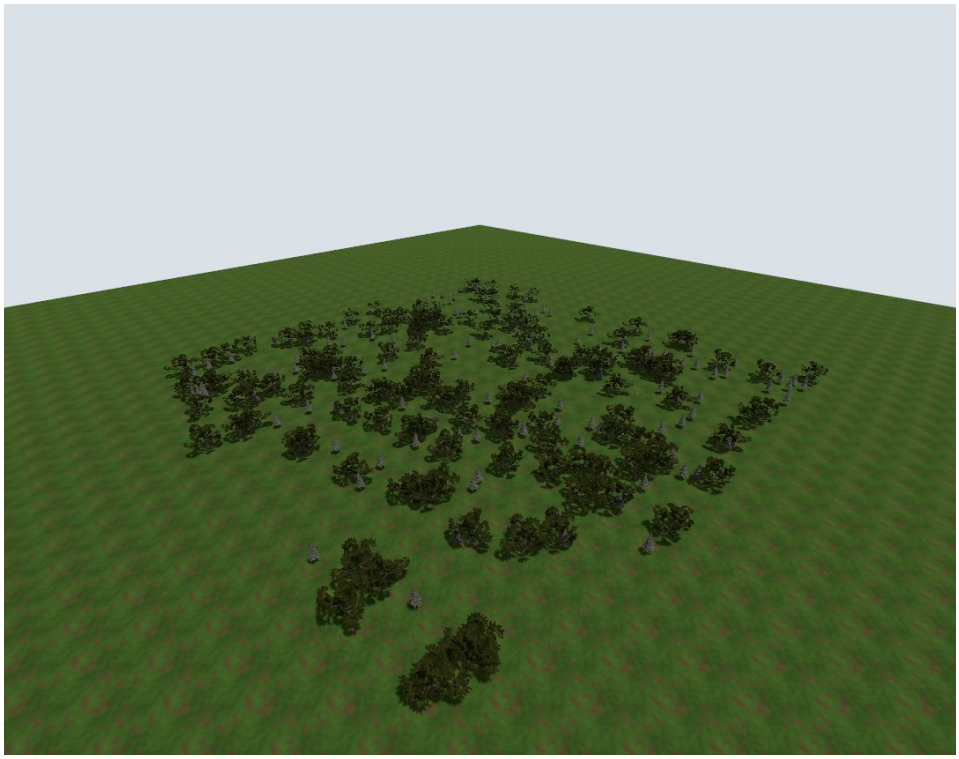


Figura 4.1: El bosque de la simulación (oclusión media) visto desde lejos.

4.3. SDF: cómo se describe un robot

Robots y mundos se escriben en **SDF** (XML): eslabones con masa, inercia, forma visual y forma de colisión; articulaciones; sensores; plugins. El Tarot 650 del proyecto se arma con las mallas, la masa y los motores del Tarot, los sensores del x500 de PX4, la cámara bajo los patines, viento habilitado y un LiDAR (capítulo 14).

4.4. Personas con etiqueta

Gazebo puede asignar una **etiqueta** a cada modelo (sistema `Label`). Las personas llevan la etiqueta 1. Una cámara especial de Gazebo (*boundingbox camera*) devuelve entonces el recuadro de la parte *visible* de cada persona: así se etiquetaron solas las 3000 imágenes del dataset del detector (capítulo 8).

4.5. Tiempo simulado

La simulación tiene su propio reloj: si la computadora es lenta, todo va más despacio pero los resultados no cambian. El **factor de tiempo real** mide la proporción. Todos los nodos usan el reloj de Gazebo (`use_sim_time`).

4.6. SITL: el piloto real en la computadora

En **SITL** (*software in the loop*) el mismo programa del piloto automático que volaría en el dron real —PX4 o ArduCopter— corre en la computadora y recibe los sensores simulados. Lo que se prueba aquí es casi lo que volará. El paso siguiente sería HITL: el controlador de vuelo físico conectado a la simulación.

Problema encontrado y cómo se resolvió

PX4 trae su propia configuración de Gazebo, que carga un sistema de video (`GstCameraSystem`). Si el codificador se atora, acumula cuadros de la cámara: llegó a 110 MB/s, 23 GB en minutos, y el sistema operativo terminó matando la simulación (y otra que estaba abierta). Ahora el *launch* abre Gazebo sin esa configuración y conecta PX4 a ese Gazebo (`PX4_GZ_STANDALONE`).

Capítulo 5

ROS 2 y PX4 para principiantes

En este capítulo

La terminal en cinco minutos, qué es ROS 2 (nodos, tópicos, mensajes, parámetros, launch), qué es PX4, y cómo se hablan los dos por medio del agente uXRCE-DDS.

5.1. La terminal en cinco minutos

```
pwd                     # ?`en qué carpeta estoy?
ls                      # ?`qué hay aquí?

cd ~/ros2_ws            # entrar a una carpeta

cat archivo.txt         # ver un archivo
Ctrl+C                  # detener el programa que está corriendo
```

Consejo

Tab completa nombres y la flecha arriba recupera la orden anterior.

5.2. ROS 2

ROS 2 (*Robot Operating System*) conecta muchos programas pequeños. Cada programa es un **nodo**; los nodos se mandan **mensajes** por canales con nombre llamados **tópicos**. Uno *publica*, otros se *suscriben*, sin conocerse entre sí.

Ejemplo de la vida diaria

Una cocina de restaurante: quien toma pedidos los pega en un tablero (tópico), el cocinero los toma de ahí y deja los platos en la barra (otro tópico), de donde los recoge el mesero. Nadie habla con nadie directamente.

En este proyecto:

Tópico	Qué lleva
`/camera`	imágenes de la cámara del dron (Gazebo → ROS)
`/detecciones`	recuadros de personas que encuentra YOLO
`/personas/estimaciones`	una posición en el suelo por cada detección
`/personas/candidatas`	candidatas fusionadas con su confianza y número de detecciones
`/personas/gps`, `/personas/alerta`	aviso de cada persona confirmada
`/fmu/out/vehicle_odometry`	posición y orientación estimadas por PX4
`/fmu/in/trajectory_setpoint`	consigna de posición que manda la misión a PX4
`/dron/lidar/points`	nube de puntos del LiDAR

```
ros2 node list
```

```
ros2 topic list
```

```
ros2 topic echo /personas/alerta
```

```
ros2 topic hz /camera
```

Los ajustes de cada nodo son **parámetros**; un archivo **launch** arranca todos los nodos con sus parámetros. El nuestro, `simulacion.launch.py`, también genera el bosque y arranca Gazebo, PX4 o ArduPilot y MAVProxy.

5.3. PX4

PX4 es uno de los dos pilotos automáticos de código abierto más usados (el otro es ArduPilot). Estima el estado con un filtro de Kalman (EKF2), controla en cascada (posición → velocidad → actitud → velocidad angular) y tiene modos de vuelo. Dos son clave aquí:

Offboard: PX4 obedece consignas de una computadora externa (nuestra misión) que debe llegar al menos dos veces por segundo; si dejan de llegar, PX4 se protege.

Neural Control (módulo `mc_nn_control`, en la versión 1.17): una red neuronal calcula directamente los comandos de los cuatro motores (capítulo 11).

5.4. El puente: uXRCE-DDS

PX4 tiene su propio sistema interno de mensajes (uORB). El **agente Micro XRCE-DDS** los traduce a tópicos de ROS 2 con el prefijo `/fmu/`: `/fmu/out/...` lo que publica PX4, `/fmu/in/...` lo que recibe. Los tipos de mensaje vienen del paquete `px4_msgs`, que debe ser de la misma versión que PX4 (`release/1.17`).

Problema encontrado y cómo se resolvió

En PX4 1.17 algunos mensajes son *versionados*: el estado del vehículo se publica en `/fmu/out/vehicle_status_v1`, no en `/fmu/out/vehicle_status` como decía el diseño original. Escuchar el tópico equivocado no da error: simplemente nunca llega nada.

5.5. Workspace y compilación

```
cd ~/ros2_ws
```

```
colcon build --symlink-install
source install/setup.bash
```

El workspace contiene `mision` (Python), `mision_gz_plugins` (C++), `px4_msgs` y, para ArduPilot, `ardupilot_gazebo`.

Parte II

Ver y ubicar: percepción y estimación

Capítulo 6

Qué es una imagen para una computadora

En este capítulo

Píxeles y canales de color, cómo forma una imagen una cámara, qué es el campo de visión, qué ve una cámara que apunta hacia abajo, y por qué una persona a 30 m ocupa tan pocos píxeles.

6.1. Una imagen es una tabla de números

La cámara del dron entrega imágenes de 1280 columnas por 960 filas: más de 1.2 millones de **píxeles**. Cada píxel tiene tres números entre 0 y 255: cuánto rojo, cuánto verde y cuánto azul (RGB). Una persona vista desde arriba es un pequeño grupo de píxeles de colores distintos al pasto: la ropa, la piel, el cabello.

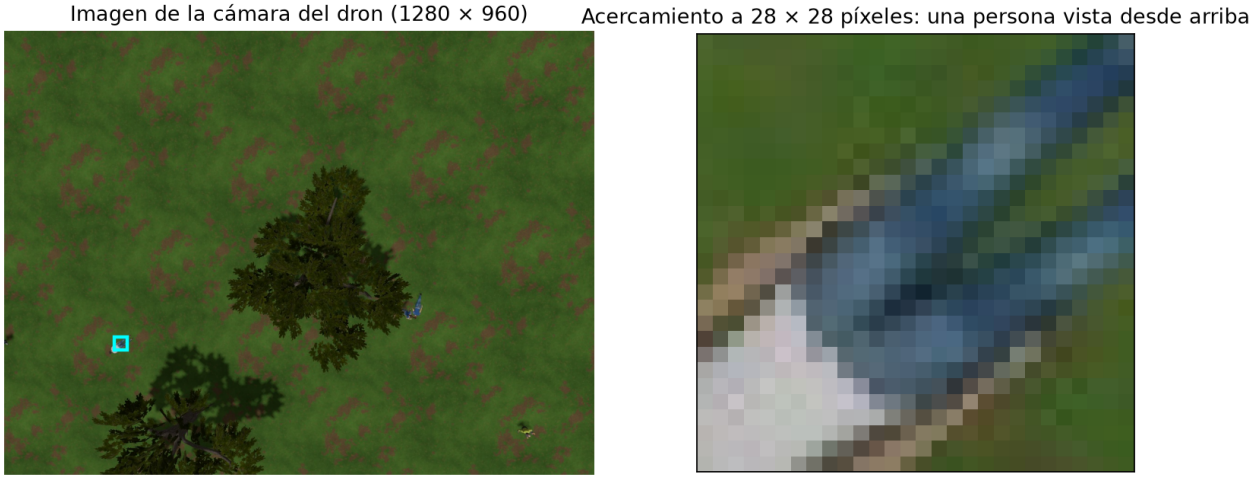


Figura 6.1: Una imagen de la cámara del dron y un acercamiento a una de las personas: a 30 m, una persona apenas ocupa unas decenas de píxeles.

6.2. Cómo forma una imagen una cámara

El modelo de **cámara estenopeica** supone una caja con un agujero: la luz de cada punto entra en línea recta y llega a un punto del sensor. Un punto a distancia Z y desplazado X aparece en

$$u = f\,\frac{X}{Z}, \qquad f = \frac{W/2}{\tan(\mathrm{HFOV}/2)} = \frac{640}{\tan(0.87)} \approx 540 \text{ píxeles.} \tag{6.1}$$

Toda la información de la cámara se resume en la **matriz intrínseca**

$$K = \begin{bmatrix} f & 0 & W/2 \\ 0 & f & H/2 \\ 0 & 0 & 1 \end{bmatrix}, \tag{6.2}$$

que Gazebo publica en `/camera_info`.

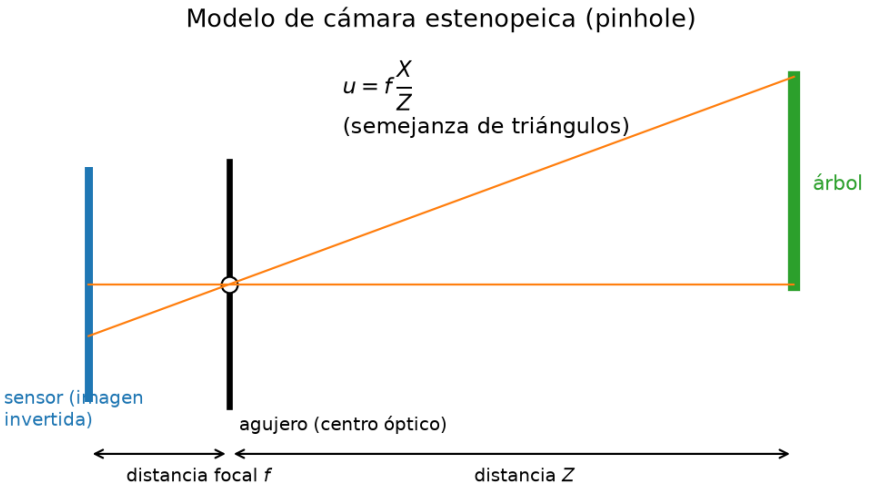


Figura 6.2: El modelo estenopeico: la imagen se forma por rayos rectos que pasan por el centro óptico.

6.3. Una cámara que mira hacia abajo

La cámara del Tarot 650 (la `mono_cam` de PX4) tiene un campo horizontal de 1.74 rad (100°) y apunta al suelo. A 30 m de altura su **huella** en el suelo mide

$$2 \cdot 30 \cdot \tan(0.87) \approx 71 \text{ m de ancho} \quad \times \quad 53 \text{ m de alto.} \tag{6.3}$$

Cada píxel cubre entonces unos $71/1280 \approx 5.5$ cm. Una persona de pie, vista desde arriba (hombros de unos 45 cm), ocupa ≈8 píxeles de ancho; acostada, unos 30 de largo. Por eso hace falta un detector entrenado para objetos pequeños y por eso la misión baja a 20 m para mirar mejor los casos dudosos.

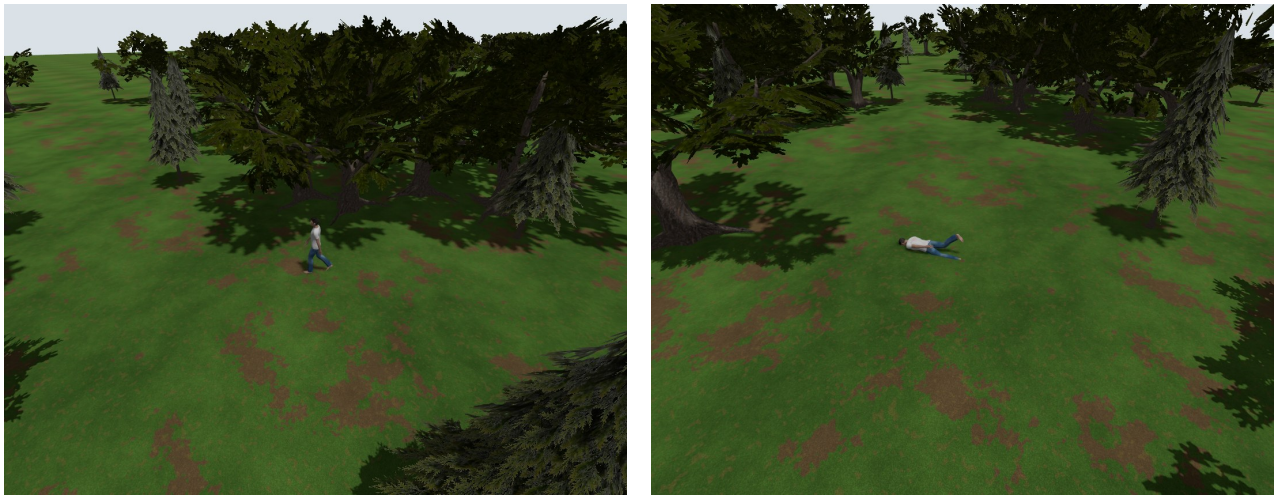


Figura 6.3: Una persona de pie y una acostada en el bosque simulado (vistas oblicuas para el libro; el dron las ve desde arriba).

6.4. Del píxel al mundo

Un píxel no dice a qué distancia está lo que muestra, solo en qué *dirección*: cada píxel es un **rayo**. Para ubicar a la persona hay que cortar ese rayo con el suelo; si además se conoce la incertidumbre de la posición y la orientación del dron, se puede calcular qué tan grande es el error (capítulo 9).

Recuerda

Una cámara da direcciones, no distancias. Con el suelo plano y la altura del dron, cada dirección se convierte en un punto del mapa.

Capítulo 7

Redes neuronales y YOLO sin complicaciones

En este capítulo

Qué es aprender con ejemplos, cómo funciona una neurona, qué es una convolución, cómo detecta objetos YOLO y cómo se mide si funciona: IoU, precisión, recall y mAP.

7.1. Programar contra aprender

Escribir reglas para reconocer a una persona desde el aire ("un grupo de píxeles de color piel...") falla de inmediato: hay ropa de todos colores, sombras, personas acostadas o tapadas por ramas. El **aprendizaje automático** hace lo contrario: se le dan miles de imágenes con la respuesta correcta y el programa ajusta sus parámetros hasta acertar.

7.2. La neurona artificial

Una neurona multiplica cada entrada por un **peso**, suma todo con un **sesgo** y pasa el resultado por una función de activación:

$$y = \text{activación}\Big(\sum_i w_i x_i + b\Big). \tag{7.1}$$

Muchas neuronas en capas forman una **red**. Entrenar es encontrar los pesos que hacen que la red acierte. La misma idea sirve para ver (YOLO) y para controlar (la red 15-64-32-4 del capítulo 11).

7.3. Convoluciones

Las redes para imágenes usan **convoluciones**: filtros pequeños que se deslizan por la imagen buscando patrones locales (bordes, texturas, formas). Las primeras capas detectan bordes; las siguientes, partes; las últimas, objetos completos. Los filtros no se diseñan: se aprenden.

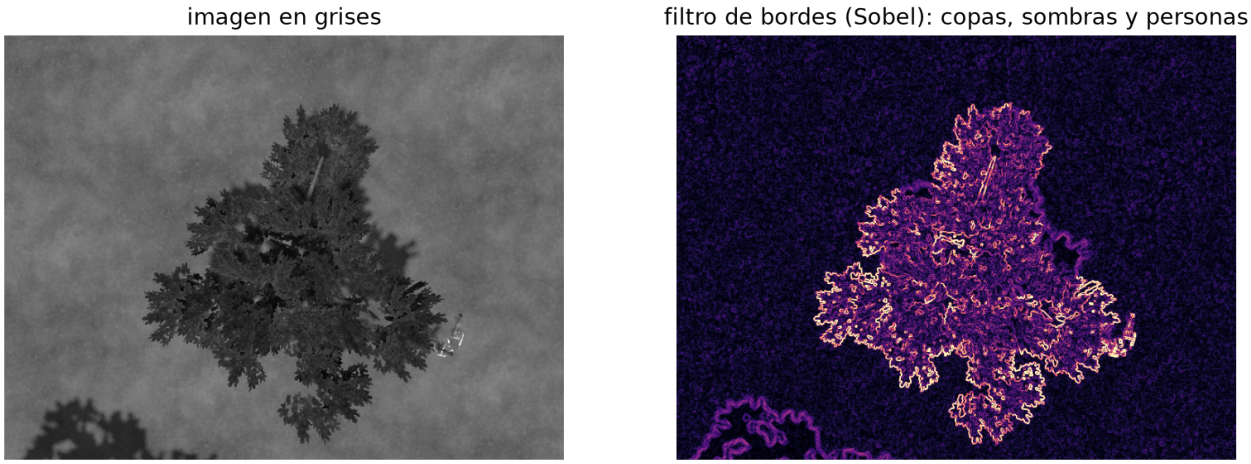


Figura 7.1: Un filtro de bordes aplicado a una imagen del dron: resalta copas, sombras y personas.

7.4. YOLO

YOLO (*You Only Look Once*) predice en una sola pasada todos los recuadros de la imagen, cada uno con su clase y su confianza. Usamos **YOLO26s**, la versión vigente de Ultralytics en tamaño *small*: la *nano* es más rápida pero pierde personas pequeñas. Procesa una imagen de 1280 píxeles en ≈16 ms en la GPU.

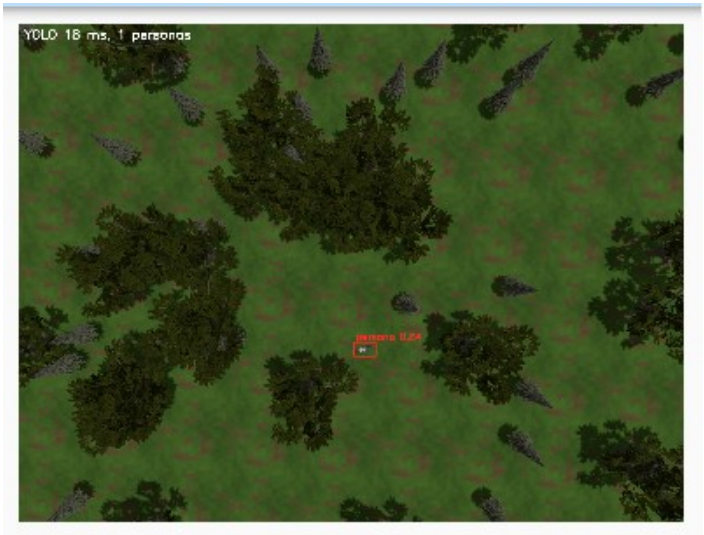


Figura 7.2: Lo que entrega YOLO durante el vuelo: el recuadro rojo con la clase “persona” y su confianza.

7.5. Cómo se mide si funciona

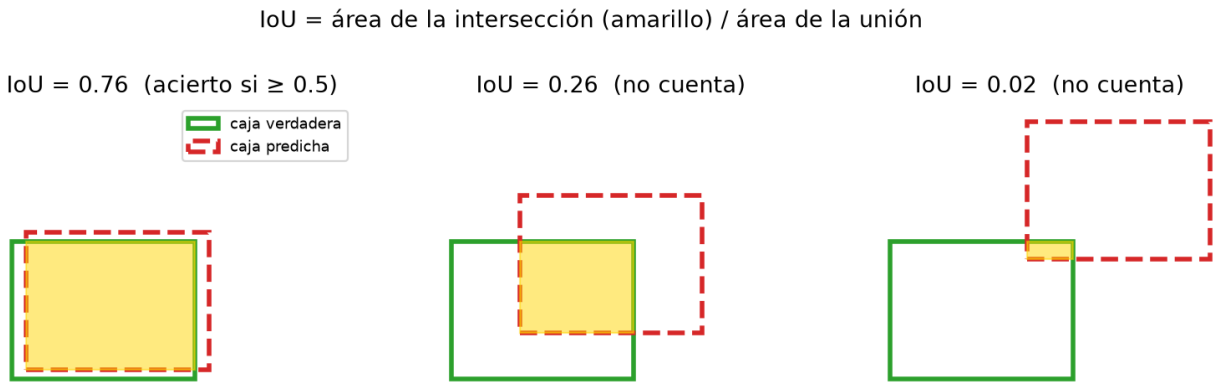


Figura 7.3: IoU: intersección sobre unión de la caja predicha y la verdadera.

Verdadero positivo: dijo "persona" y había una (con IoU ≥ 0.5).

Falso positivo: dijo "persona" y no había (por ejemplo, la sombra de un roble).

Falso negativo: había una persona y no la vio.

$$\text{precisión} = \frac{VP}{VP + FP} \qquad \text{(cuando dice "persona", ¿cuántas veces acierta?)} \tag{7.2}$$

$$\text{recall} = \frac{VP}{VP + FN} \qquad \text{(de las personas que hay, ¿cuántas encuentra?)} \tag{7.3}$$

La **mAP50** resume la curva precisión-recall con IoU ≥ 0.5; la **mAP50-95** exige además cajas más ajustadas.

Ejemplo de la vida diaria

En una búsqueda, perder a una persona (falso negativo) es mucho peor que revisar una sombra (falso positivo). Por eso el umbral de confianza del detector es bajo (0.25) y la misión decide después, juntando varias detecciones, qué candidatas son confiables.

Capítulo 8

El detector de personas

En este capítulo

Cómo se generó un dataset de 3000 imágenes etiquetadas sin dibujar un solo recuadro a mano, cómo se entrenó YOLO26s, qué tan bien quedó en cada condición y cómo funciona el nodo detector durante el vuelo.

8.1. Etiquetas automáticas

`scripts/capturar_dataset.py` genera bosques con distintos niveles de oclusión e iluminación, cada uno con 30 personas, y coloca dos cámaras con la misma óptica que la del dron (1280×960, 1.74 rad, hacia abajo): una normal y una **cámara de cajas** de Gazebo, que entrega el recuadro de la parte *visible* de cada objeto con etiqueta. Mueve las cámaras a poses al azar de un dron buscando (entre 12 y 45 m de altura, un poco inclinadas, cualquier orientación) y guarda la imagen y sus cajas. Una persona tapada por una copa sale recortada o no sale: exactamente lo que el detector puede ver.

```
python3 scripts/capturar_dataset.py ~/datasets/personas_bosque 3000
```

Resultado: 3000 imágenes con 13 202 personas, y un archivo `estratos.csv` con la oclusión, la iluminación y la altura de cada imagen.

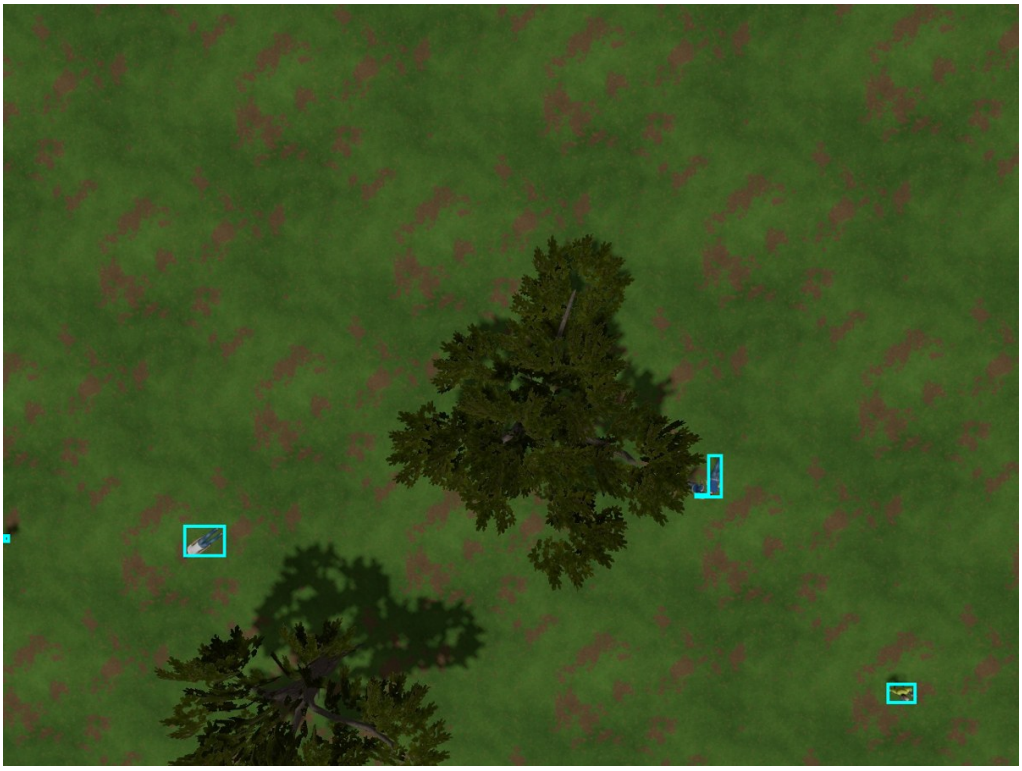


Figura 8.1: Una imagen del dataset con sus etiquetas automáticas (recuadros cian).

8.2. Entrenamiento

```
~/.venvs/yolo/bin/python scripts/entrenar_detector.py entrenar --datos ~/datasets/personas_bosque/
    datos.yaml --epocas 80
```

Se parte de los pesos de YOLO26s preentrenados en COCO y se ajusta con imágenes de 1280 píxeles (sin reducir: las personas son pequeñas) en la GPU. El mejor modelo se copia a `yolo/personas_yolo26s.pt`, que es el que usa la misión. Resultado en validación (300 imágenes): **AP50 = 0.954**, AP50-95 = 0.692, precisión 0.96, recall 0.92, 15.6 ms por imagen.

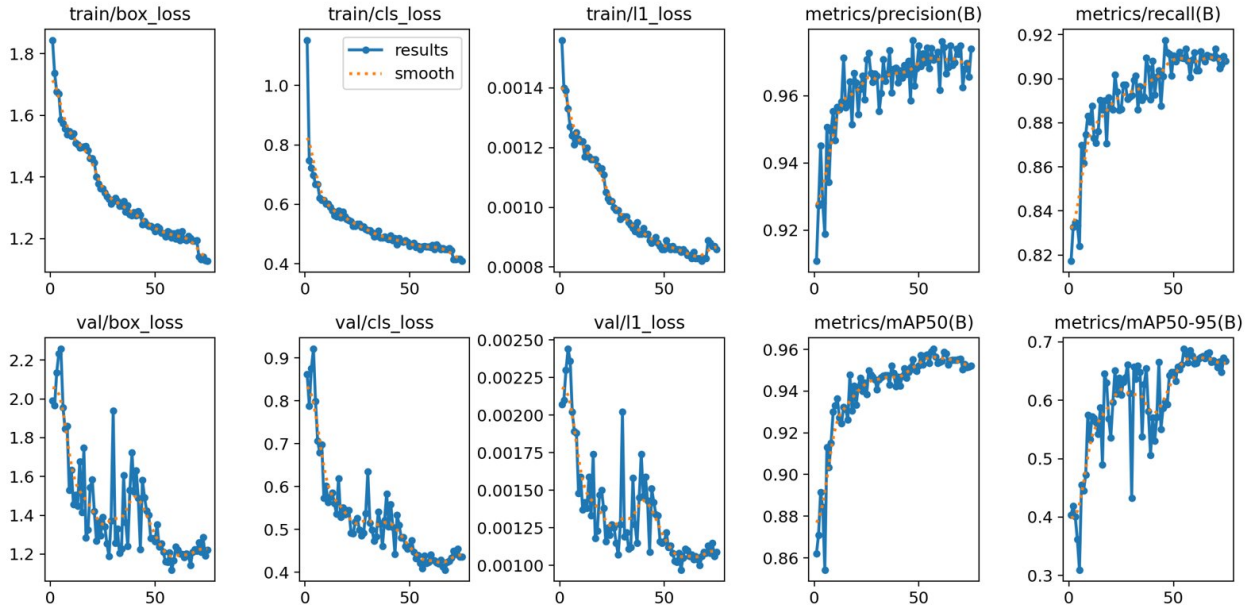


Figura 8.2: Curvas de entrenamiento del detector de personas.

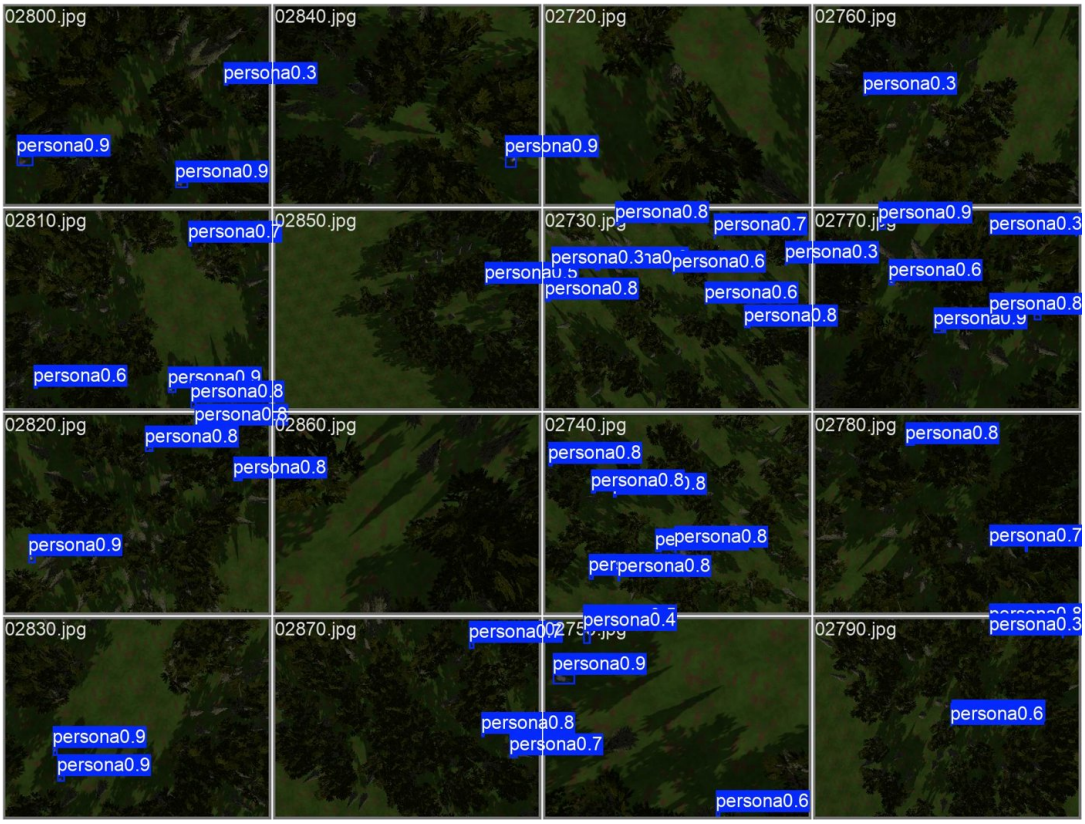


Figura 8.3: Predicciones del detector sobre imágenes de validación.

8.3. Por estrato: dónde falla

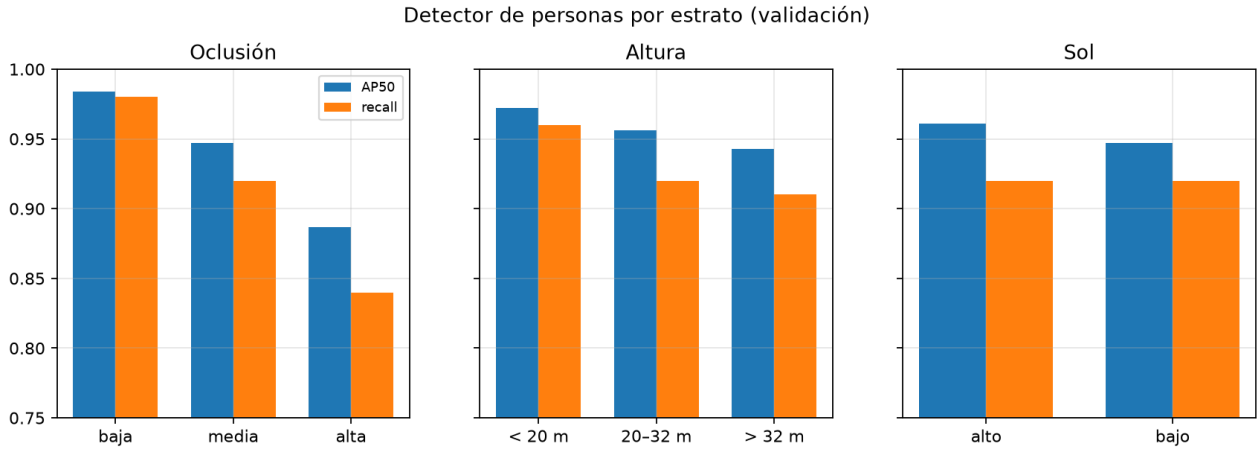


Figura 8.4: El detector pierde más personas con más copas (recall 0.98 → 0.84) y con más altura; la iluminación casi no importa.

Nota

Con el YOLO26n preentrenado en COCO, sin ajustar, la misión localizó 0 de 3 personas: COCO casi no tiene personas vistas desde arriba a 30 m. El ajuste con imágenes del simulador es indispensable; para el dron real habrá que agregar imágenes reales (por ejemplo, el conjunto HERIDAL; `scripts/preparar_datos.py` las convierte).

8.4. El nodo detector

`mision/detector_node.py` corre con el Python de `~/.venvs/yolo`, lee `/camera`, aplica el modelo y publica las cajas en `/detecciones` (con la marca de tiempo de la imagen) y la imagen anotada en `/camera_yolo`, que Gazebo muestra en el panel "Detección de personas". Para estudiar qué pasa si el detector es más lento (por ejemplo en una computadora a bordo), se le puede agregar latencia (`latencia_extra_ms`) o hacer que salte cuadros (`saltar_cuadros`).

Capítulo 9

Ubicar a la persona: localización y fusión

En este capítulo

Cómo se convierte un recuadro de la imagen en un punto del suelo, de dónde viene el error y cómo se calcula su tamaño (propagación de covarianza), y cómo se juntan muchas detecciones de la misma persona en una sola candidata cada vez más precisa y confiable.

9.1. Del recuadro al suelo: el rayo

Para cada detección se toma el centro del recuadro (u, v) —con la cámara hacia abajo, la persona se ve casi desde arriba y el centro es el mejor punto de apoyo—. Su dirección en el marco de la cámara es $K^{-1}(u, v, 1)^\top$; se pasa al mundo con la orientación de la cámara (R_{BC}, fija respecto al cuerpo) y la del dron (R, que estima PX4):

$$r = R\,R_{BC}\,K^{-1}\begin{bmatrix} u \\ v \\ 1 \end{bmatrix}, \qquad p_c = \xi + R\,t_{BC}, \tag{9.1}$$

donde ξ es la posición del dron y $t_{BC} = (0, 0, -0.24)$ m la de la cámara en el Tarot 650 (bajo los patines; en el x500, -0.25 m). El punto del suelo ($z = z_t = 0$) es

$$x = p_c + \lambda\, r, \qquad \lambda = \frac{z_t - p_{c,z}}{r_z}. \tag{9.2}$$

Si el rayo no baja ($r_z \geq 0$) no hay intersección; si forma más de 60° con la vertical, el error sería enorme y se descarta (ver el recuadro de abajo).

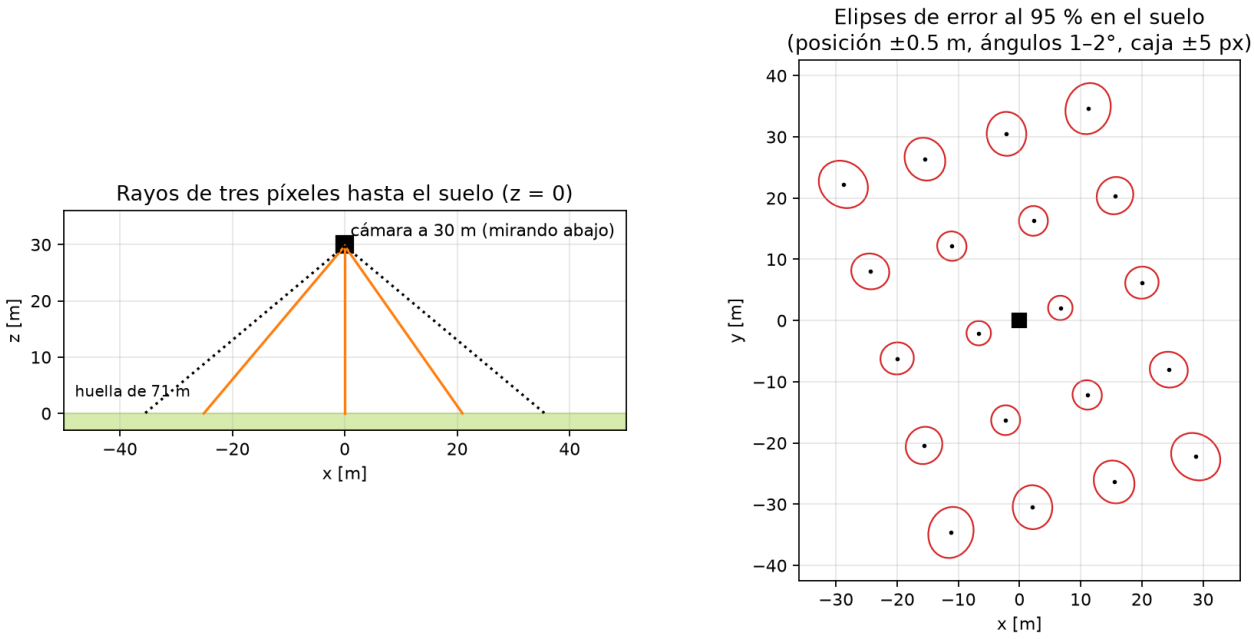


Figura 9.1: Izquierda: rayos de tres píxeles hasta el suelo. Derecha: elipses de error al 95 % para una rejilla de píxeles, con la incertidumbre típica de PX4.

9.2. ¿Qué tan grande es el error?

El punto depende de ocho cantidades inciertas: la posición del dron (3), su orientación (3) y el centro del recuadro (2). Si cada una tiene una varianza conocida (Σ_p, Σ_η, Σ_u), a primer orden la covarianza del punto es

$$\Sigma_x \approx J_p \Sigma_p J_p^\top + J_\eta \Sigma_\eta J_\eta^\top + J_u \Sigma_u J_u^\top , \tag{9.3}$$

con J las derivadas (jacobianos) de x respecto a cada grupo, calculadas numéricamente. La elipse de la figura 9.1 es la representación de Σ_x. PX4 publica las varianzas de su estimación en `/fmu/out/vehicle_odometry`, así que el localizador las usa en cada detección.

Ejemplo de la vida diaria

Apuntar con un láser desde un balcón: un pequeño temblor de la mano (orientación) mueve el punto más cuanto más lejos está el suelo; moverse uno mismo medio metro (posición) mueve el punto medio metro, esté donde esté. Por eso el error por orientación crece con la altura y el de posición no.

`codigo/sim_localizacion.py` valida la fórmula con miles de simulaciones al azar (Monte Carlo) y separa las tres fuentes:

Caso	Altura	RMSE	Posición	Orientación	Detección
nominal	20 m	0.85 m	0.92	0.31	0.26
nominal	30 m	0.94 m	0.92	0.47	0.39
nominal	40 m	1.05 m	0.92	0.62	0.52
posición degradada ($\sigma_p = 2$ m)	30 m	3.02 m	3.34	0.47	0.39
orientación degradada ($\sigma_\eta = 2°$)	30 m	1.81 m	0.92	1.86	0.39
detección degradada ($\sigma_u = 20$ px)	30 m	1.78 m	0.92	0.47	1.57

Tabla 9.1: Error de localización y su desglose (salida de `sim_localizacion.py`; las tres últimas columnas son la contribución lineal en metros). Nominal: $\sigma_p = 0.5$ m, $\sigma_\eta = 0.5°$, $\sigma_u = 5$ px. Con un GPS normal domina la posición del dron.

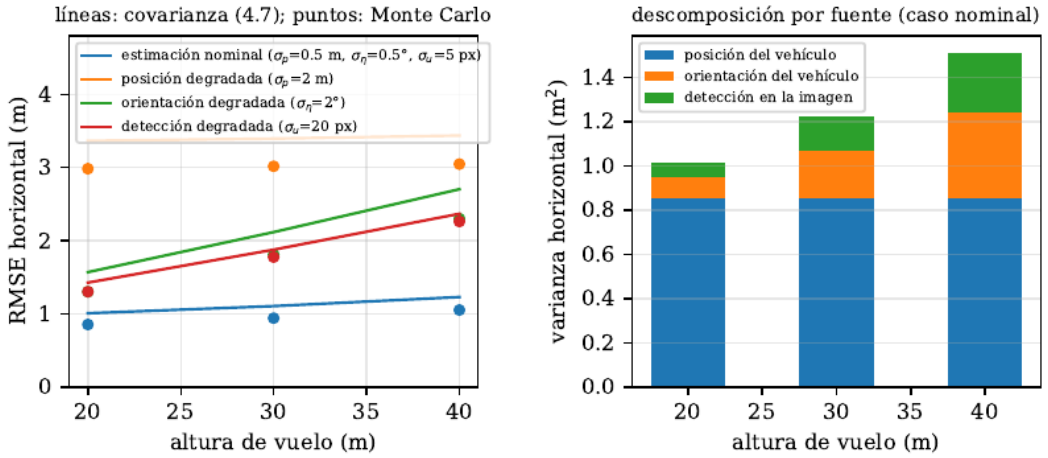


Figura 9.2: Figura de `sim_localizacion.py`: la fórmula lineal y el Monte Carlo coinciden.

9.3. Juntar detecciones: asociación y fusión

A 3 m/s, una persona queda en la imagen varios segundos y el detector la ve decenas de veces. Cada detección produce un punto con su elipse. `localizador_node` mantiene una lista de **candidatas** y, para cada detección nueva:

1. **Asociación:** calcula su distancia de **Mahalanobis** a cada candidata,

$$d^2 = (x - c)^\top (\Sigma_x + \Sigma_c)^{-1} (x - c), \tag{9.4}$$

una distancia "medida en elipses". Si la menor es menor que 9.21 (el percentil 99 % de una χ^2 con 2 grados de libertad), es la misma persona; si no, nace una candidata nueva.

2. **Fusión:** combina las dos estimaciones pesando cada una por su precisión (fusión de información):

$$\Sigma_c' = \left(\Sigma_c^{-1} + \Sigma_x^{-1}\right)^{-1}, \qquad c' = \Sigma_c'\left(\Sigma_c^{-1} c + \Sigma_x^{-1} x\right). \tag{9.5}$$

La candidata nueva c' queda más cerca de la estimación más precisa, y su elipse Σ_c' es siempre más chica que las dos anteriores.

3. **Confianza:** acumula la confianza del detector como la probabilidad de que al menos una detección sea correcta: $C \leftarrow 1 - (1 - C)(1 - c_i)$.

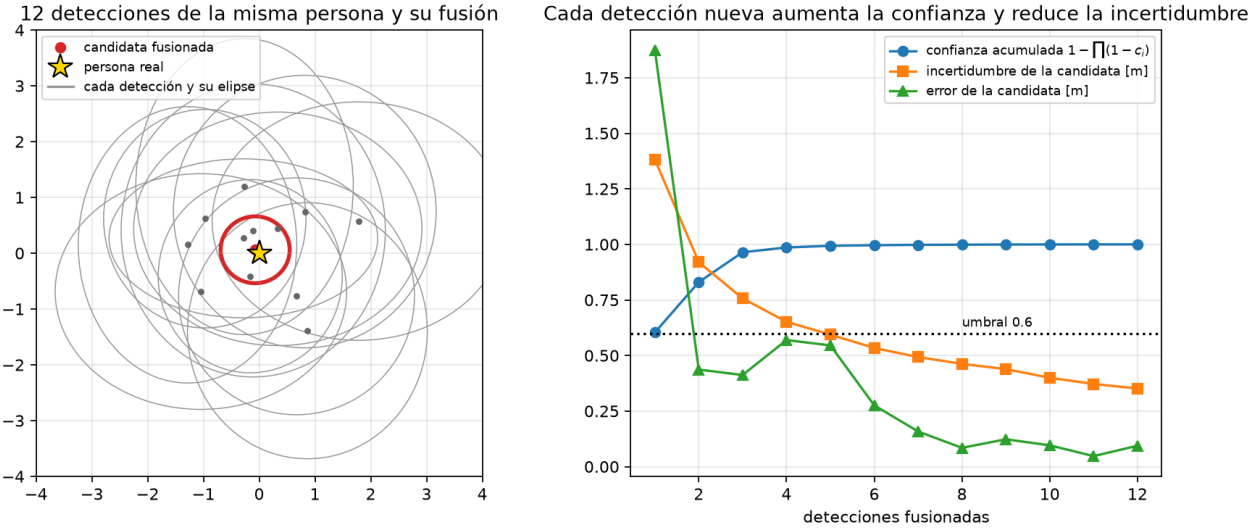


Figura 9.3: Doce detecciones simuladas de una persona: cada una con su elipse (gris) y la candidata fusionada (rojo). Con cada detección la confianza sube y la incertidumbre baja.

La misión considera **detectada** a una candidata con confianza acumulada ≥ 0.6, y una persona **localizada** si alguna detectada queda a menos de 5 m (asignación uno a uno). Las detectadas que no corresponden a nadie son **falsas alarmas**.

Problema encontrado y cómo se resolvió

La confianza acumulada sube muy rápido: dos detecciones de 0.5 ya dan 0.75. Por eso una sombra vista dos veces ya cuenta como candidata confirmada. En las pruebas, exigir además al menos 10 detecciones fusionadas bajó las falsas alarmas de 28–31 a 4–5 por vuelo sin perder personas; los avisos GPS usan ese criterio. Las falsas alarmas que quedan son persistentes: grupos de robles que el detector confunde una y otra vez.

Problema encontrado y cómo se resolvió

En las vueltas de la ruta el dron se inclina; la cámara, que mira hacia abajo, alcanzaba a ver cerca del horizonte, y un rayo casi horizontal cortaba el suelo a cientos de metros: salían candidatas a 250 m, fuera del bosque. El localizador ahora descarta rayos a más de 60° de la vertical (con el dron nivelado las esquinas de la imagen quedan a ≈57°, así que en vuelo normal no se pierde nada).

Problema encontrado y cómo se resolvió

PX4 marca sus mensajes con el reloj de la computadora y la cámara con el de Gazebo. Emparejar cada imagen con la pose del dron en ese instante exige conocer el desfase; el localizador lo estima con el mínimo de (llegada − marca) en tiempo simulado.

Parte III

El control: de la cascada a la red neuronal

Capítulo 10

Control automático desde cero

En este capítulo

Qué es controlar, la diferencia entre lazo abierto y lazo cerrado, el error, los controladores P, PD y PID, cómo se lee una respuesta y por qué un dron necesita control en cascada.

10.1. ¿Qué es controlar?

Controlar es conseguir que algo se comporte como queremos aunque haya perturbaciones. Queremos que el dron esté a 30 m de altura; el viento lo empuja hacia abajo; el controlador acelera los motores para compensar. Al "algo" que se controla se le llama **planta** (aquí, el dron); a lo que queremos, **referencia** o **consigna**; a lo que nos estorba, **perturbación**.

10.2. Lazo abierto y lazo cerrado

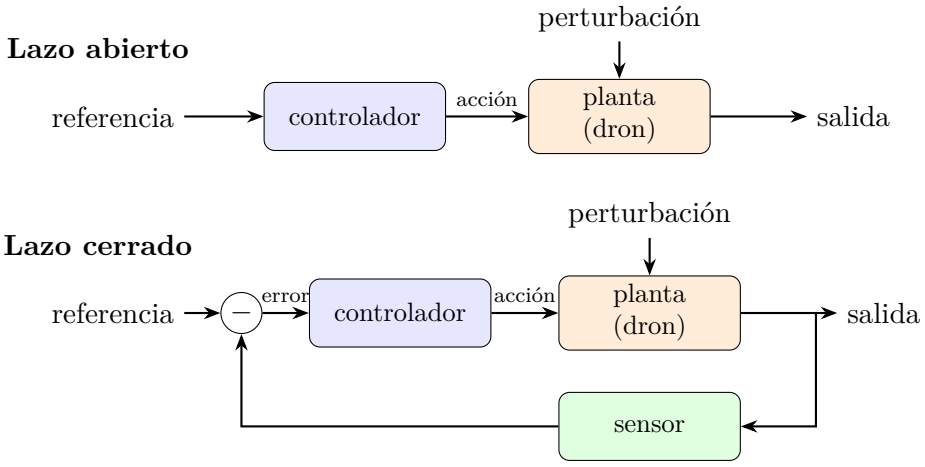


Figura 10.1: En lazo abierto se actúa "a ciegas". En lazo cerrado se mide el resultado, se compara con lo que se quiere y se corrige: eso es la *retroalimentación*.

Ejemplo de la vida diaria

Una tostadora es lazo abierto: tuesta un tiempo fijo, aunque el pan se queme. Ajustar la temperatura de la regadera es lazo cerrado: sientes el agua (sensor), la comparas con la que quieres (error) y mueves la llave (acción). Si alguien abre otra llave en la casa (perturbación), lo notas y corriges.

Un dron sin lazo cerrado no se puede volar: cualquier ráfaga o diferencia entre motores lo voltea en un segundo. El **error** es la diferencia entre lo que queremos y lo que hay, $e = \text{referencia} - \text{medición}$, y todo controlador hace algo con él.

10.3. El controlador proporcional (P)

La idea más simple: actuar en proporción al error. Si estoy 2 m abajo, empujo el doble que si estoy 1 m abajo:

$$u = K_p\, e. \tag{10.1}$$

K_p es la **ganancia proporcional**. El problema con un dron: el control P actúa sobre la *aceleración*, y un objeto que se acelera hacia el objetivo llega con velocidad, se pasa, regresa, se vuelve a pasar... oscila para siempre (curva gris de la figura 10.2).

10.4. El término derivativo (D): frenar a tiempo

Hay que frenar antes de llegar. El término derivativo actúa sobre la *velocidad* del error: si me acerco rápido, freno.

$$u = K_p\, e + K_d\, \dot{e}. \tag{10.2}$$

Es como un amortiguador. Con poco K_d oscila; con mucho, llega muy lento; hay un punto intermedio (figura 10.2, derecha).

10.5. El término integral (I): no rendirse

Si un viento constante empuja hacia abajo, el PD llega a un equilibrio un poco por debajo del objetivo: con ese pequeño error ya compensa el viento. El término integral *acumula* el error en el tiempo: mientras haya error, sigue creciendo hasta eliminarlo.

$$u = K_p\, e + K_d\, \dot{e} + K_i \int e\, dt. \tag{10.3}$$

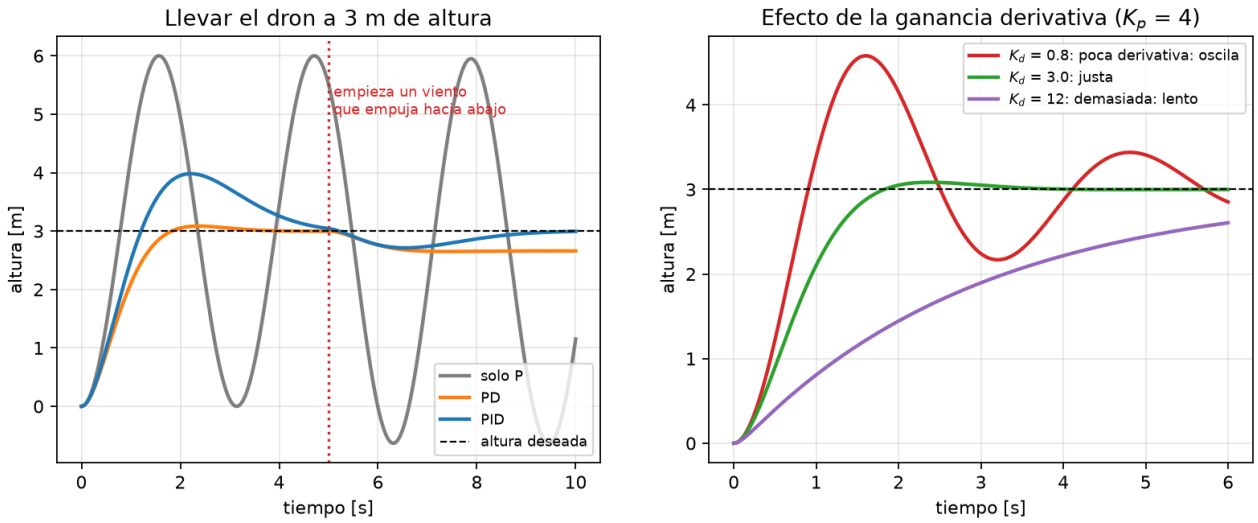


Figura 10.2: Simulación de la altura de una masa de 2.2 kg (como un dron mediano). Izquierda: solo P oscila; PD llega pero el viento (desde $t = 5$ s) le deja un error; PID lo corrige. Derecha: efecto de K_d.

Recuerda

P empuja hacia el objetivo, **D** frena para no pasarse, **I** corrige lo que queda. El PID es, de lejos, el controlador más usado en el mundo, incluidos PX4 y ArduPilot.

10.6. Cómo se lee una respuesta

Para comparar controladores se les pide un cambio brusco de referencia (un **escalón**) y se mide:

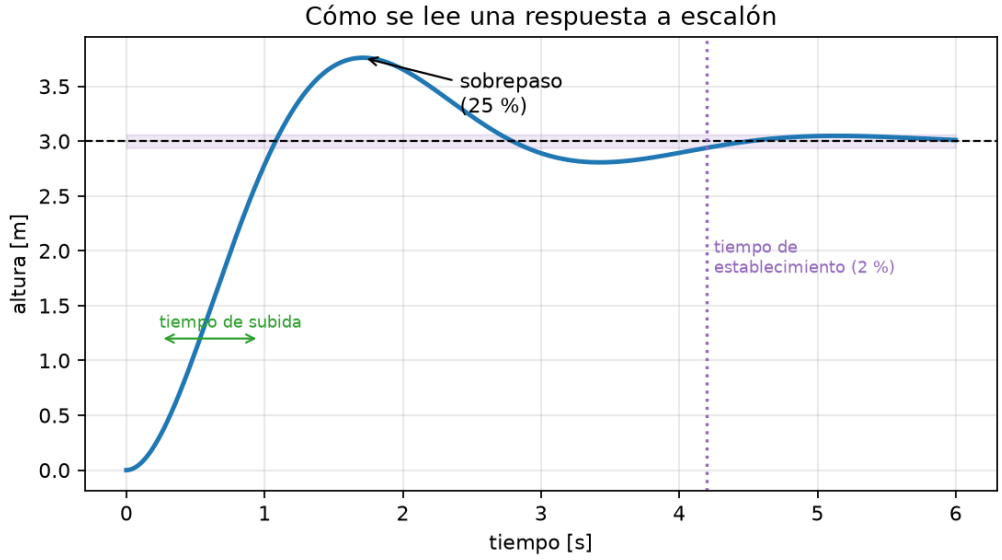


Figura 10.3: Medidas típicas de una respuesta a escalón.

Sobrepaso: cuánto se pasa del objetivo.

Tiempo de subida: cuánto tarda en ir del 10 % al 90 % del cambio.

Tiempo de establecimiento: a partir de cuándo se queda cerca del objetivo (por ejemplo, a menos del 2 %).

Error en estado estacionario: lo que queda de error cuando todo se calma.

En este proyecto se usa el **RMSE de seguimiento**: la raíz del promedio de las distancias al cuadrado entre el dron y la ruta durante el barrido. Un solo número que resume qué tan bien sigue el dron su camino.

10.7. Control en cascada: así se controla un dron

El dron no puede "moverse a la derecha" directamente: tiene que inclinarse. Por eso hay dos lazos, uno dentro del otro:

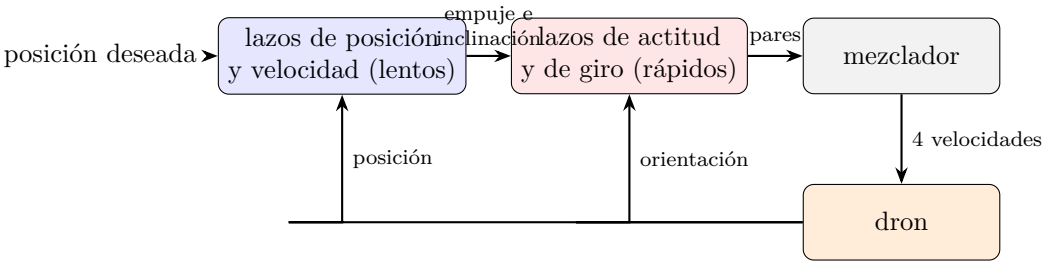


Figura 10.4: Control en cascada del cuadrirrotor.

1. El **lazo de posición** compara dónde está el dron con dónde debe estar y calcula qué fuerza total necesita: cuánto empuje y hacia dónde apuntarlo.
2. Apuntar el empuje significa inclinar el dron: el **lazo de actitud** calcula los pares para alcanzar esa inclinación. Es mucho más rápido que el de posición.
3. El **mezclador** convierte empuje y pares en la velocidad de cada motor.

PX4 tiene cuatro lazos en cascada (posición → velocidad → actitud → velocidad angular), cada uno un P o un PID, que corren a cientos de hertz; ArduPilot hace lo mismo con otros nombres. Ese es el **controlador en cascada** del proyecto. La alternativa es reemplazar toda la cascada por una sola red neuronal que, a partir del estado del dron y del error de posición, calcula directamente los cuatro motores: el tema del próximo capítulo.

Pruébalo tú

Compara la cascada con la red neuronal en la misma misión, sin interfaz:

```
ros2 launch mision simulacion.launch.py gui:=false controlador:=cascada salida:=$HOME/cascada.
    json
ros2 launch mision simulacion.launch.py gui:=false controlador:=red salida:=$HOME/red.json
```

En cada JSON, `rmse_seguimiento` es el error de seguimiento de la ruta y `duracion_s` lo que tardó.

Capítulo 11

Una red neuronal que vuela el dron

En este capítulo

Qué significa que una red neuronal controle los motores, cómo aprende imitando a un experto (clonación y DAgger), qué es el aprendizaje por refuerzo (PPO), cómo se mete la red en PX4, qué problemas tuvo la primera versión y cómo se corrigieron.

11.1. La idea

La cascada de PX4 son cuatro controladores encadenados, con decenas de ganancias ajustadas a mano. La alternativa es **una sola red neuronal** que reciba el estado del dron y el error de posición y calcule directamente los cuatro motores. PX4 1.17 trae un módulo para eso, `mc_nn_control` (*Neural Control*), con una interfaz fija:

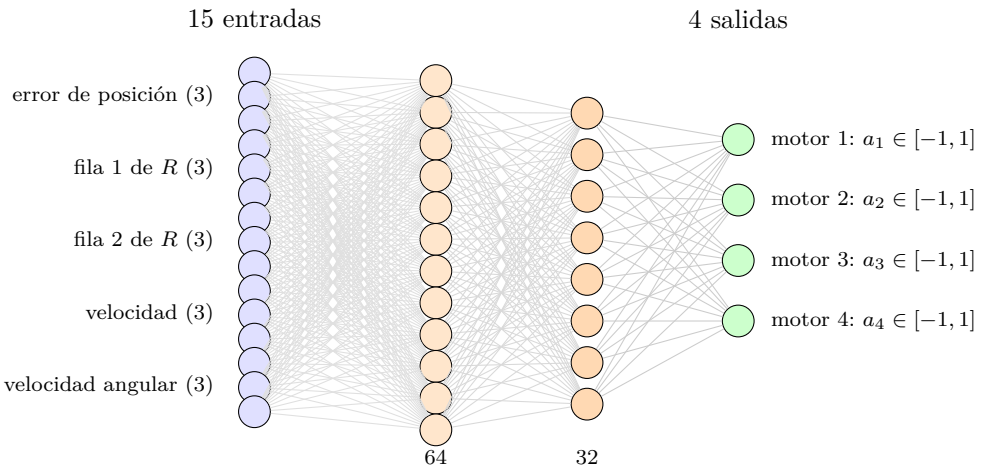


Figura 11.1: La red 15-64-32-4 con ReLU: 3 236 pesos. Entradas en convención ENU-FLU; cada salida se convierte en la velocidad del motor $\omega_i = \omega_{max}\sqrt{(a_i + 1)/2}$.

Recuerda

La red no tiene memoria: en cada instante solo ve el estado actual y el error de posición. No puede "acumular" el error como el término integral de un PID. Eso tendrá consecuencias.

11.2. Aprender imitando: clonación de comportamiento y DAgger

¿Cómo se entrena? Lo más directo es **imitar a un experto**: se vuela con un controlador que ya funciona (una cascada PD, el "maestro"), se registran miles de pares (estado, acciones del maestro) y se entrena la red para que, ante cada estado, dé las mismas acciones. Eso es **clonación de comportamiento**, un problema de regresión como cualquier otro.

Ejemplo de la vida diaria

Aprender a manejar viendo a un instructor desde el asiento del copiloto. El problema: el instructor nunca se sale del carril, así que tú nunca ves cómo se regresa al carril. El día que te desvíes un poco, no sabrás qué hacer.

DAgger (*Dataset Aggregation*) resuelve eso: después de la primera ronda, vuela *la red*, se registran los estados *a los que ella llega* (incluidos sus errores) y el maestro dice qué habría hecho en cada uno. Se agregan esos ejemplos y se reentrena. Tres rondas bastan. Como el entrenamiento es en un simulador en Python (el modelo del capítulo de física, a 400 Hz), se agrega además **aleatorización de dominio**: en cada episodio la masa, las inercias, k_f, el retardo de los motores y el viento cambian un poco, para que la red no dependa de valores exactos.

Para el Tarot 650 de la simulación (2.25 kg; la cámara corre el centro de masa 0.67 mm en x y en y):

```
cd codigo
```

```
python3 nn_imitacion.py --vehiculo tarot650 --masa 2.25 --com 0.00067 0.00067 --adelanto \
    --salida pesos_nn_tarot650_busqueda.npz --figura ../imagenes/nn_imitacion_tarot650_busqueda.pdf
```

`--vehiculo` elige la física del simulador de entrenamiento (masa, inercias, brazos, k_f, k_m, ω_{max} y retardos de los motores) de la tabla `VEHICULOS` del mismo programa; con `--vehiculox500` se entrena la red del x500 (capítulo 14).

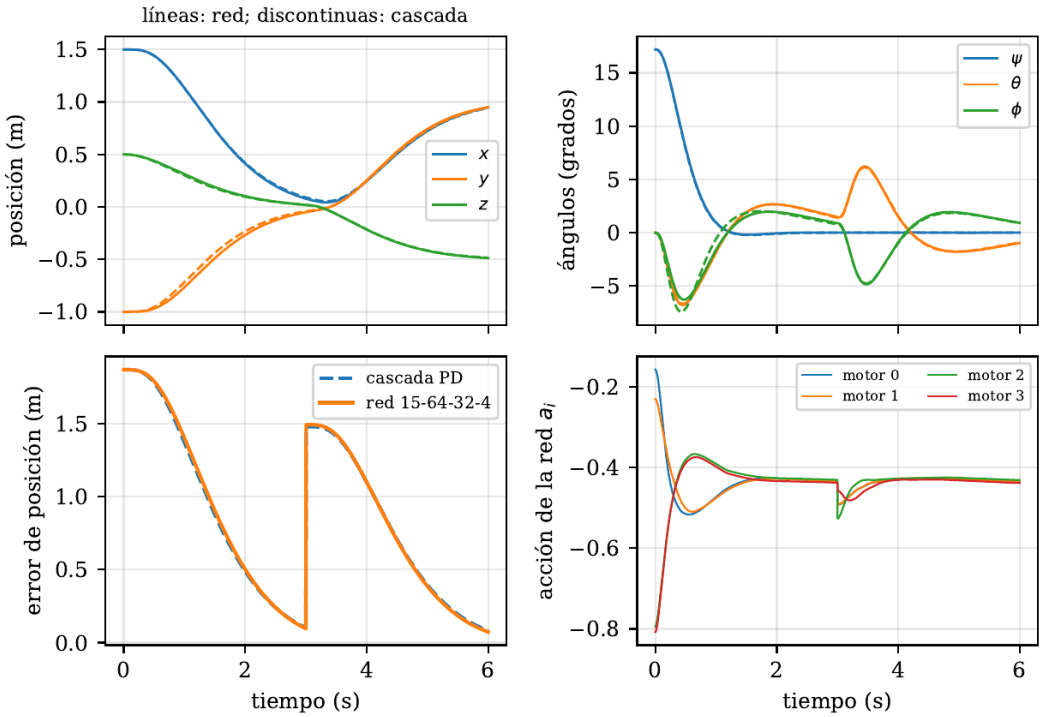


Figura 11.2: La red del Tarot 650 (líneas continuas) y el maestro (discontinuas) ante los mismos escalones. En vuelo estacionario cada motor da $a_i \approx -0.44$: $\omega_i = 680\sqrt{(a_i+1)/2} \approx 361$ rad/s.

11.3. Aprender por refuerzo: PPO

Imitar tiene un techo: la red será, a lo más, tan buena como el maestro. El **aprendizaje por refuerzo** quita al maestro: la red prueba acciones, recibe una **recompensa** (alta si está cerca del objetivo, baja si se aleja o se estrella) y ajusta sus pesos para aumentar la recompensa acumulada. **PPO** (*Proximal Policy Optimization*) es el algoritmo más usado: cambia la política a pasos pequeños para no destruir lo aprendido. Lleva dos redes: el *actor* (la que vuela) y el *crítico* (que estima qué tan bueno es cada estado).

La idea del proyecto es partir de la red de imitación y refinarla con PPO (`codigo/ppo_refinamiento.py`).

11.4. De Python a PX4

1. `codigo/exportar_tflite.py` convierte los pesos a **TensorFlow Lite** (32 bits), comprueba que solo use las operaciones que tiene el módulo (capas totalmente conectadas, ReLU y suma) y que dé lo mismo que numpy (diferencia de $\sim 10^{-6}$), y genera el arreglo en C `control_net.cpp/.hpp` (14 936 bytes).
2. `scripts/instalar_px4.sh` lo copia a `mc_nn_control` y recompila `px4_sitl_neural`.
3. La misión cambia al modo *Neural Control* al llegar a la altura de trabajo y regresa a offboard antes de aterrizar.

Con ArduPilot, que no tiene un módulo equivalente, la misma red corre en el nodo `controlador_red` y el plugin `SelectorMotores` deja pasar sus comandos en lugar de los de ArduPilot (capítulo 18).

Problema encontrado y cómo se resolvió

El diseño original tenía dos detalles que impedían que la red volara: el parámetro `MC_NN_THRST_COEF` de PX4 trabaja en revoluciones por segundo, no por minuto, y faltaban `MC_NN_MIN_RPM` y `THR_MDL_FAC`. Con ellos mal, la misma salida de la red producía otro empuje. Los valores correctos están en el *airframe* `px4/4100_gz_tarot650_busqueda` (para $\omega_{max} = 680$ rad/s, `MC_NN_THRST_COEF` $= 8\pi^2/\omega_{max}^2 \times 10^5 = 17.0754$, `MC_NN_MIN_RPM` 955 y `MC_NN_MAX_RPM` 6494; los del x500 en `px4/4099_gz_x500_busqueda`) y una prueba los verifica recorriendo toda la cadena de PX4 (red → mezclador → ESC).

11.5. Tres problemas de la primera versión y su solución

La primera versión de la red se entrenó y se probó con el x500, el vehículo de las primeras corridas (capítulo 14); por eso las cifras de esta sección son del x500. Las tres soluciones se aplican igual a la red del Tarot 650.

11.5.1. Un sesgo de 20 cm

En la primera prueba en SITL quedaba un error permanente en altura de −0.27 m: la red se había entrenado con $m = 2.0$ kg (la masa del cuerpo del x500) y el x500 completo pesa 2.114 kg. Sin integral, $\Delta z = \Delta m\, g/(m\, k_p)$. Con la masa correcta el error en altura desapareció, pero quedó un sesgo horizontal de ≈0.2 m: la cámara corre 0.7 mm el centro de masa y la red, que nunca vio ese desequilibrio, se quedaba inclinada ≈1°. La solución fue usar la geometría real en el simulador de entrenamiento (`--com`): el maestro compensa con los motores y la red aprende a hacerlo. Por eso la red del Tarot se entrena con su masa completa (`--masa2.25`) y su centro de masa corrido (`--com0.000670.00067`).

11.5.2. La red iba a la mitad de velocidad

Una ley PD sin prealimentación, siguiendo una referencia que se mueve a velocidad v, se queda atrás $\frac{k_d}{k_p}v$: con las ganancias del maestro, 4.8 m a 3 m/s. La misión no deja que la consigna se adelante más de 3 m, así que la red avanzaba a ≈1.6 m/s y la misión duraba casi el doble.

La solución clásica de control: mandar una **consigna adelantada**

$$p_{cons} = p_{ref} + \frac{k_d}{k_p}\, v_{ref} \quad \Rightarrow \quad k_p(p_{cons} - p) - k_d v = k_p(p_{ref} - p) + k_d(v_{ref} - v), \tag{11.1}$$

que convierte a la PD en una PD con prealimentación de velocidad, sin cambiar la interfaz de la red. Pero la red tiene que haber visto esos errores de varios metros: se reentrenó con `--adelanto` (la mitad de los episodios con referencias que avanzan hasta 4 m/s).

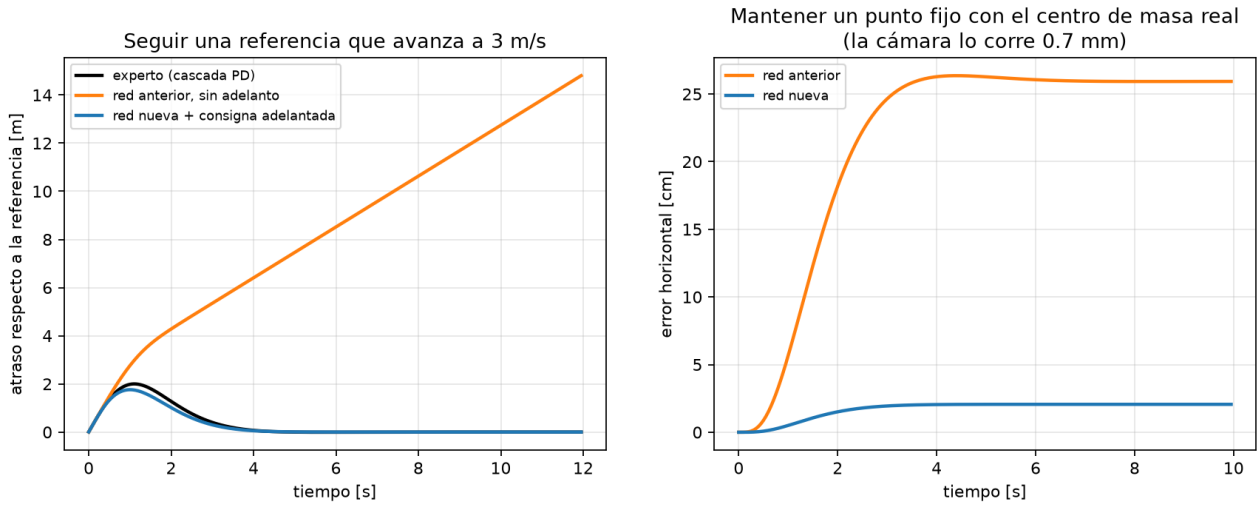


Figura 11.3: Izquierda: siguiendo una referencia a 3 m/s, la red anterior se atrasa sin parar; la nueva, con la consigna adelantada, sigue igual que el maestro. Derecha: con el centro de masa real, la red anterior se queda a 26 cm del punto; la nueva, a 2 cm.

11.5.3. PPO divergía

En la primera versión, PPO estaba a la par de la imitación hasta la iteración ∼100 y luego se deterioraba hasta estrellarse en 4 de 40 pruebas. La causa principal estaba en la recompensa original: cada paso valía $-(|e|^2 + \dots) + 1$, negativo con más de ∼1 m de error (hasta −25 a 5 m), mientras que estrellarse costaba solo −10 una vez. ¡Terminar pronto "convenía"! Además los cortes de episodio por tiempo se trataban como si el dron hubiera fallado, y el crítico, que empieza al azar, desordenaba la política de imitación en las primeras iteraciones.

La versión mejorada (`--mejorado`) usa una recompensa acotada y positiva mientras el dron vuele, penaliza estrellarse con −100, trata bien los cortes por tiempo, calienta el crítico 15 iteraciones antes de tocar la política, frena los cambios demasiado grandes (KL) y guarda solo la mejor política.

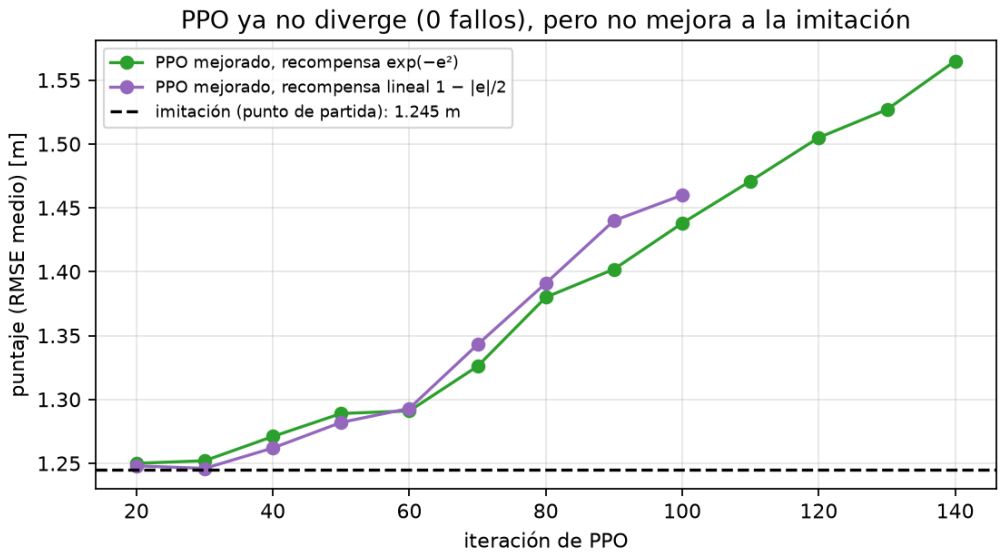


Figura 11.4: Con las correcciones PPO ya no diverge (ningún fallo), pero con ninguna de las dos recompensas probadas mejora a la imitación: se usa la red de imitación.

Nota

Un resultado negativo también es un resultado. Que PPO no mejore aquí es razonable: la red no tiene memoria y el maestro ya es una buena PD; buena parte del error que queda viene del viento y la masa aleatorios, que ninguna ley sin integral puede eliminar. Una dirección para mejorarla sería darle a la red una entrada con el error acumulado, lo que exige modificar el módulo de PX4.

11.6. Resultado en la misión

Piloto	Controlador	Localizadas	Error medio	RMSE ruta	Duración
PX4	cascada	3/3	0.61 m	0.07 m	238 s
PX4	red anterior	3/3	0.71 m	0.11 m	427 s
PX4	red nueva	3/3	0.52 m	0.23 m	**211 s**
ArduPilot	red anterior	3/3	0.46 m	0.12 m	451 s
ArduPilot	red nueva	3/3	0.47 m	0.19 m	**240 s**

Tabla 11.1: La misma misión (bosque medio, sol alto, sin viento, 3 personas). Con la red nueva la misión dura la mitad, igual o menos que con la cascada; sigue la ruta algo peor en las esquinas, donde la consigna adelantada cambia de dirección de golpe. Corridas con el x500 del 23 de octubre.

Con el Tarot 650, su red y ArduPilot volando con el EKF3 (24 de octubre, el mismo escenario):

Piloto	Controlador	Localizadas	Error medio	RMSE ruta	Duración
PX4	cascada	3/3	0.54 m	0.08 m	248 s
PX4	red	3/3	0.49 m	0.27 m	**213 s**
ArduPilot	cascada	3/3	0.65 m	0.10 m	337 s
ArduPilot	red	3/3	0.71 m	0.29 m	**269 s**

Tabla 11.2: La misión con el Tarot 650. Con la red dura de 20 a 37 % menos que con la cascada.

Nota

Las primeras corridas del Tarot con la red daban 0.39–0.44 m de RMSE. La red no tenía la culpa: la misión cambiaba de punto de ruta cuando el *dron* llegaba a 1.5 m de la esquina. Con la cascada el dron va atrás de la consigna y ella llega primero; con la red y la consigna adelantada el dron va a la par, así que la consigna, todavía a 1 m de la esquina, se iba en diagonal al siguiente punto, recortaba la esquina y el dron quedaba corrido medio metro durante todo el tramo siguiente. Ahora el punto cambia cuando el dron está cerca **y** la consigna ya llegó. Moraleja: antes de culpar al controlador, compara lo que hizo el dron con lo que se le pidió.

Parte IV

Construir la simulación

Capítulo 12

Instalación paso a paso

En este capítulo

Qué hay que instalar y en qué orden: ROS 2 y Gazebo, PX4 con el módulo neuronal, el agente uXRCE-DDS, el workspace, ArduPilot y los entornos de Python. Todo está en `scripts/instalar_entorno.sh`, pensado para correrse bloque por bloque y sin `sudo` salvo donde se indica.

12.1. 0. ROS 2 Jazzy y Gazebo Harmonic

Ubuntu 24.04 con ROS 2 Jazzy y el puente `ros-jazzy-ros-gz` (que trae Gazebo Harmonic), instalados con el repositorio oficial de ROS 2 (`ros-jazzy-desktop`, `ros-dev-tools`, `ros-jazzy-ros-gz`).

12.2. 1. PX4 release/1.17

```
cd ~ && git clone --recursive -b release/1.17 https://github.com/PX4/PX4-Autopilot.git
```

```
python3 -m venv --system-site-packages ~/.venvs/px4
~/.venvs/px4/bin/pip install -r ~/PX4-Autopilot/Tools/setup/requirements.txt ninja
```

Cuidado

La etiqueta `v1.17.0` de PX4 no existe (solo `v1.17.0-rc2`): se usa la rama `release/1.17`. Ubuntu 24.04 no deja instalar con `pipinstall--user` sobre el Python del sistema (PEP 668): por eso cada grupo de dependencias va en su propio entorno.

12.3. 2. Agente Micro XRCE-DDS 2.4.3

```
cd ~ && git clone -b v2.4.3 https://github.com/eProsima/Micro-XRCE-DDS-Agent.git
```

```
cd Micro-XRCE-DDS-Agent && mkdir build && cd build
```

```
env -i HOME=$HOME PATH=/usr/bin:/bin cmake .. -DCMAKE_INSTALL_PREFIX=$HOME/.local
env -i HOME=$HOME PATH=/usr/bin:/bin make -j"$(nproc)" && make install
```

Se compila con un entorno limpio (`env -i`) para que use su propio Fast-DDS y no el de ROS, y se instala en `~/.local` sin `sudo`.

12.4. 3. Workspace

```
git clone https://github.com/jairoroke/<repositorio>.git ~/ros2_ws
cd ~/ros2_ws/src
git clone -b release/1.17 https://github.com/PX4/px4_msgs.git
git clone https://github.com/ArduPilot/ardupilot_gazebo.git       # solo para piloto:=ardupilot
cd ~/ros2_ws && colcon build --symlink-install && source install/setup.bash
```

12.5. 4. Entornos de Python

```
python3 -m venv ~/venv-mision
~/venv-mision/bin/pip install numpy scipy matplotlib pillow tensorflow-cpu    # exportar la red
```

- `~/venv-mision`: TensorFlow, para exportar la red a TFLite.
- `~/.venvs/yolo`: Ultralytics con PyTorch y CUDA (y `numpy<2`, para leer los mensajes de ROS 2). Corre el detector y el refinamiento PPO.
- `~/venv-ardupilot`: MAVProxy, pymavlink y `python-xlib` (para acomodar las ventanas).

12.6. 5. Vehículo, red y compilación de PX4

```
cd ~/ros2_ws/src/mision
python3 scripts/generar_vehiculo.py      # tarot650_busqueda, x500_busqueda y sus variantes para
    ArduPilot
cd codigo
python3 nn_imitacion.py --vehiculo tarot650 --masa 2.25 --com 0.00067 0.00067 --adelanto \
    --salida pesos_nn_tarot650_busqueda.npz --figura ../imagenes/nn_imitacion_tarot650_busqueda.pdf
~/venv-mision/bin/python exportar_tflite.py --pesos pesos_nn_tarot650_busqueda.npz --salida
    control_net_tarot650_busqueda
bash ../scripts/instalar_px4.sh ~/ros2_ws/src/mision/codigo/control_net_tarot650_busqueda
```

`instalar_px4.sh` copia a `~/PX4-Autopilot` los vehículos `tarot650_busqueda` y `x500_busqueda`, los *airframes* `4100_gz_tarot650_busqueda` y `4099_gz_x500_busqueda` (carpeta `px4/`) y la red (guardando una vez la original como `control_net.*.px4`), agrega `/fmu/out/neural_control` a los

tópicos que exporta PX4 y compila `makepx4_sitl_neural`. Repite estos pasos cada vez que cambies la red o el vehículo.

Cuidado

PX4 lleva **una sola red compilada**: ahora es la del Tarot. Para volar el x500 con `controlador:=red` en PX4 hay que reinstalar `control_net_x500_busqueda`. Con ArduPilot no hace falta: `controlador_red` carga los pesos del vehículo que se lanza.

12.7. 6. ArduPilot (opcional)

ArduCopter 4.7.2 compilado para SITL en `~/ardupilot` (`./waf configure --board sitl \&\& ./waf copter`) y el plugin `ardupilot_gazebo` en el workspace.

12.8. Comprobación

```
cd ~/ros2_ws/src/mision && python3 -m pytest -q test/          # 8 pruebas
ros2 launch mision simulacion.launch.py gui:=false             # una misión completa, unos 4 min
```

Capítulo 13

Dónde está cada cosa

```
~/ros2_ws/src/
|-- mision/                    paquete principal (Python)
|   |-- mision/                nodos de ROS 2 y bibliotecas
|   |-- codigo/                programas de referencia y la red neuronal
|   |-- launch/simulacion.launch.py
|   |-- scripts/               generadores, dataset, entrenamiento, campaña, instalación
|   |-- models/                tarot650_busqueda, x500_busqueda, personas, árboles, suelo
|   |-- px4/                   airframes 4100_gz_tarot650_busqueda y 4099_gz_x500_busqueda
|   |-- config/                GUI de Gazebo, parámetros de ArduPilot (tarot650.parm, x500.parm)
|   |-- yolo/                  personas_yolo26s.pt (detector)
|   `-- test/                  pruebas (pytest)
|-- mision_gz_plugins/         SelectorMotores (C++), para ArduPilot con la red
|-- px4_msgs/                  mensajes de PX4 (clonado, release/1.17)
`-- ardupilot_gazebo/          plugin de ArduPilot (clonado)
```

13.1. Nodos y módulos

Archivo	Qué hace
`detector_node.py`	YOLO sobre `/camera`; publica `/detecciones` y la imagen anotada.
`localizador_node.py`	Rayo–terreno, covarianza, asociación de Mahalanobis y fusión.
`mision_node.py`	Barrido offboard, re-observación, Neural Control, avisos GPS, evasión, métricas.
`mision_ardupilot.py`	La misma misión con ArduCopter (GUIDED por MAVLink).
`controlador_red.py`	La red como controlador de motores con ArduPilot.
`geometria.py`	Proyección, localización y covarianza (compartidas con las pruebas).
`metricas.py`	Personas localizadas, falsas alarmas, error, distancia a la ruta.
`evasion.py`, `geo.py`	Evasión con el LiDAR; conversión ENU → GPS.
`mavproxy_trayectoria.py`, `ventanas_sitl.py`	Mapa de MAVProxy con ruta y personas; acomodo de ventanas.
`modo_neural.py`, `prueba_neural.py`	Activar Neural Control a mano; escalones de prueba.

13.2. Programas de referencia (`codigo/`)

`sim_modelo.py`	Modelo del x500 con el control base (sigue usando el x500).
`sim_localizacion.py`	Localización, covarianza y Monte Carlo.
`nn_imitacion.py`	Red 15-64-32-4 por clonación + DAgger (`--vehiculotarot650|x500`, `--masa`, `--com`, `--adelanto`).
`ppo_refinamiento.py`	Refinamiento por PPO (`--mejorado`, `--forma`).
`exportar_tflite.py`	TFLite y `control_net.cpp/.hpp`.

Sin las opciones nuevas, estos programas reproducen exactamente las cifras de referencia.

13.3. Scripts

`generar_mundo.py`	Bosque por oclusión, iluminación, viento y semilla, con el JSON de las personas verdaderas.
`generar_vehiculo.py`	El Tarot 650 (y el x500) con cámara bajo los patines, viento y LiDAR, para PX4 y ArduPilot.
`capturar_dataset.py`, `entrenar_detector.py`	Dataset con cajas visibles y entrenamiento por estrato.
`correr_experimentos.py`, `analisis_wilcoxon.py`	Campaña factorial y pruebas estadísticas.
`instalar_entorno.sh`, `instalar_px4.sh`	Instalación.

Capítulo 14

El Tarot 650 en Gazebo

En este capítulo

De dónde sale el vehículo de la simulación, qué se le agregó para que lo vuele PX4, cuáles de sus datos son supuestos y cómo se cambian por los del dron real, cómo es su variante para ArduPilot y cómo volar todavía con el x500 de las primeras corridas.

14.1. El tarot650_busqueda

El vehículo es el **Tarot Ironman 650**. Su modelo de Gazebo, `tarot650_busqueda`, lo arma `scripts/generar_vehiculo.py` con dos fuentes:

- **Del Tarot:** las mallas de su CAD (placas, brazos, motores, tren de aterrizaje y hélices, carpeta `models/tarot650_busqueda/meshes`), la masa, las inercias, la posición de los rotores, las hélices y los motores. Todo sale de la tabla `VEHICULOS['tarot650']` de `cuadrirotor_sim/scripts/generar_modelo.py`, otro paquete del mismo *workspace*, así que los dos proyectos usan el mismo dron.
- **Del x500 de PX4:** los sensores (IMU, barómetro, magnetómetro y GPS, con el ruido de PX4) y con los mismos nombres, porque el puente de PX4 con Gazebo los busca así.

Además, `generar_vehiculo.py` pone los motores en el orden de PX4 (`rotor_0` adelante-derecha antihorario, `rotor_1` atrás-izquierda antihorario, `rotor_2` adelante-izquierda horario, `rotor_3` atrás-derecha horario), que es también el de las salidas 1–4 de ArduPilot en un "X", y le agrega:

1. **La cámara 24 cm debajo del centro**, justo bajo los patines, mirando hacia abajo (la `mono_cam` de PX4: 1280×960, campo horizontal de 1.74 rad).
2. **Viento habilitado** en el cuerpo (`<enable_wind>`), para que el sistema `WindEffects` del mundo lo empuje.
3. **Un LiDAR 3D** ($360° \times 120$ haces, 16 canales de $-35°$ a $+15°$, 30 m, 10 Hz, `/dron/lidar/points`) para la evasión de obstáculos.

Parámetro	Valor
Masa total	2.25 kg (cuerpo 2.12, rotores 4×20 g, cámara 50 g)
Inercia del cuerpo	diag(0.0304, 0.0321, 0.0585) kg m^2
Inercia con los rotores	diag(0.0341, 0.0361, 0.0660) kg m^2
Rotores	$(\pm 0.2198, \pm 0.212)$ m, 4 cm sobre el centro, hélices de 15”
k_f, k_m, ω_{max}	4.24×10^{-5} N/(rad/s)^2, 0.016 m, 680 rad/s
Empuje máximo	19.6 N por motor, ≈3.6 veces el peso en total
Vuelo estacionario	$\omega_h = 361$ rad/s por motor
Constantes de tiempo de los motores	0.03 s al subir, 0.05 s al bajar
Cámara	1280×960, HFOV 1.74 rad, hacia abajo, $t_{BC} = (0, 0, -0.24)$ m
Centro de masa	corrido 0.67 mm en x y en y por la cámara

Tabla 14.1: Parámetros del Tarot 650 de la simulación (`models/tarot650_busqueda/model.sdf` y la tabla `VEHICULOS` de `codigo/nn_imitacion.py`). Los de masa, inercias, k_f y motores son supuestos.

El *airframe* de PX4 es `px4/4100_gz_tarot650_busqueda`. Además de la geometría de los rotores (`CA_ROTOR*`), fija las salidas a los motores de Gazebo entre 100 y 680 rad/s (`SIM_GZ_EC_MIN`, `SIM_GZ_EC_MAX`), el empuje de vuelo estacionario (`MPC_THR_HOVER` 0.25, con `THR_MDL_FAC` 0.8) y los parámetros del modo *Neural Control* para la red del Tarot (`MC_NN_THRST_COEF` 17.0754, `MC_NN_MIN_RPM` 955, `MC_NN_MAX_RPM` 6494; capítulo 11).

Para curiosos (te lo puedes saltar)

¿De dónde sale `MPC_THR_HOVER` = 0.25? En vuelo estacionario, $\omega_h = 361$ rad/s. La salida normalizada del motor es $(361 - 100)/580 = 0.45$ y, con la curva $0.8x^2 + 0.2x$ que PX4 deshace con `THR_MDL_FAC` = 0.8, el empuje normalizado es $0.8(0.45)^2 + 0.2(0.45) \approx 0.25$.

14.2. Supuestos que hay que reemplazar

Cuidado

La masa, las inercias, k_f y los retardos de los motores del Tarot de la simulación son **supuestos**: no se midieron en un dron real. Las mallas dan la forma, no la física. Antes de llevar la red neuronal o las ganancias a un Tarot real hay que reemplazarlos por los de ese dron.

Los pasos, en orden:

1. Mide la masa (con batería, cámara y todo lo que vaya a volar), las inercias, k_f y k_m (en un banco de empuje) y la respuesta de los motores.
2. Ponlos en la tabla `VEHICULOS['tarot650']` de `cuadrirotor_sim/scripts/generar_modelo.py` y en la de `codigo/nn_imitacion.py`, y vuelve a generar el vehículo (`python3scripts/generar_vehiculo.py`).
3. Si cambian la posición de los rotores, ω_{max} o el empuje de vuelo estacionario, ajusta el *airframe* `px4/4100_gz_tarot650_busqueda` y `config/ardupilot/tarot650.parm`.
4. Reentrena la red (`nn_imitacion.py--vehiculotarot650--masa...--com...`), expórtala e instálala en PX4 (capítulo 12).

14.3. La variante para ArduPilot

`tarot650_busqueda_ardupilot` es el mismo vehículo con:

- `ArduPilotPlugin`, que intercambia sensores y motores con ArduCopter (salidas de 1000–2000 µs → 0–680 rad/s, mismo orden de motores que PX4);
- una IMU para ArduPilot, con ruido y sesgo supuestos (giróscopo 0.004 rad/s, acelerómetro 0.1 m/s^2), y `OdometryPublisher` a 400 Hz (el estado para la red);
- `no_time_sync=0`, para que ArduPilot vuele con su EKF3 y no con el estado verdadero del simulador (capítulo 18);
- el sistema `SelectorMotores` (paquete `mision_gz_plugins`), que deja pasar los comandos de ArduPilot o los de la red, según lo que pida la misión.

En esa variante el vehículo va dentro del mundo, que corre a 1 kHz (lo que espera ArduPilot). Sus parámetros de vuelo están en `config/ardupilot/tarot650.parm`: salidas de 1000–2000 µs con curva de empuje cuadrática pura (`MOT_THST_EXPO` 1.0), `MOT_THST_HOVER` 0.28 ($(361/680)^2$), lazos de tasa más suaves que los del Iris (el Tarot tiene motores más lentos) y `AHRS_EKF_TYPE` 3.

```
python3 scripts/generar_vehiculo.py      # escribe models/tarot650_busqueda, models/x500_busqueda y
     sus variantes _ardupilot
bash scripts/instalar_px4.sh ~/ros2_ws/src/mision/codigo/control_net_tarot650_busqueda    # y los
    copia a PX4
```

14.4. El x500 de PX4 sigue disponible

Las primeras corridas, las del 22 y el 23 de octubre, se hicieron con el **x500** de PX4, y varias cifras del libro vienen de ellas. El vehículo sigue en el paquete y se elige con un argumento:

```
ros2 launch mision simulacion.launch.py vehiculo:=x500
```

`x500_busqueda` parte del `x500_mono_cam_down` de PX4 con los mismos tres cambios (cámara bajo los patines, viento y LiDAR); usa el *airframe* `4099_gz_x500_busqueda`, `config/ardupilot/x500.parm` y la red `pesos_nn_x500_busqueda.npz` / `control_net_x500_busqueda`.

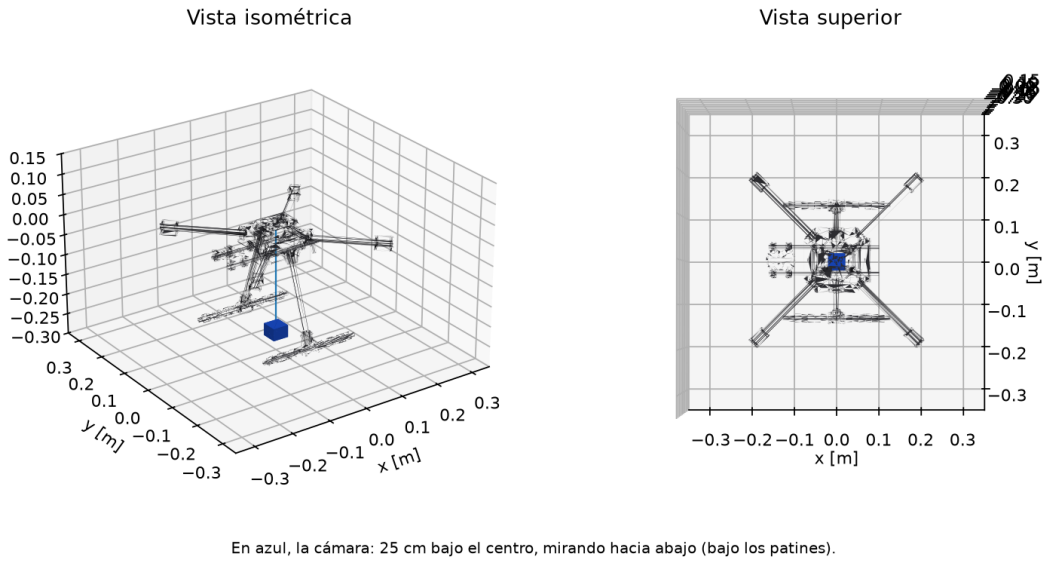


Figura 14.1: El x500 de PX4, dibujado a partir de las mallas de su modelo.

Parámetro	**Valor**
Masa total	2.114 kg (cuerpo 2.0, rotores 4×16 g, cámara 50 g)
Inercia del cuerpo	diag(0.0217, 0.0217, 0.040) kg m^2
Rotores	$(\pm 0.174, \pm 0.174)$ m, hélices de 13”
k_f, k_m, ω_{max}	8.54858×10^{-6} N/(rad/s)^2, 0.016 m, 1000 rad/s
Constantes de tiempo de los motores	12.5 ms al subir, 25 ms al bajar
Cámara	1280×960, HFOV 1.74 rad, hacia abajo, $t_{BC} = (0, 0, -0.25)$ m
Centro de masa	corrido 0.7 mm en x y en y por la cámara

Tabla 14.2: Parámetros del x500 de la simulación.

Problema encontrado y cómo se resolvió

En el modelo de PX4 la cámara está 10 cm *arriba* del centro y mira hacia abajo a través del cuerpo: el armazón, los motores y el tren de aterrizaje tapaban más de la mitad de la imagen. El diseño original suponía 10 cm *abajo*. En el x500 la cámara se movió 25 cm abajo y en el Tarot va 24 cm abajo; el localizador usa el valor de cada vehículo (t_{BC}, parámetro `z_camara` que le pasa el *launch*).

Cuidado

PX4 lleva una sola red compilada, y ahora es la del Tarot. Para volar el x500 con `controlador:=red` en PX4 hay que reinstalar la suya: `bashscripts/instalar_px4.sh~/ros2_ws/src/mision/codigo/control_net_x500_busqueda`. Con ArduPilot no hace falta: `controlador_red` carga los pesos del vehículo que se lanza.

14.5. Las personas

Cuatro modelos de OpenRobotics (Gazebo Fuel, CC0): de pie, caminando, mujer y sentada (Rescue Randy), todos estáticos y con la etiqueta 1 del sistema `Label`. La persona acostada es uno de pie girado 90°. Los copia y adapta `scripts/importar_personas.py`.

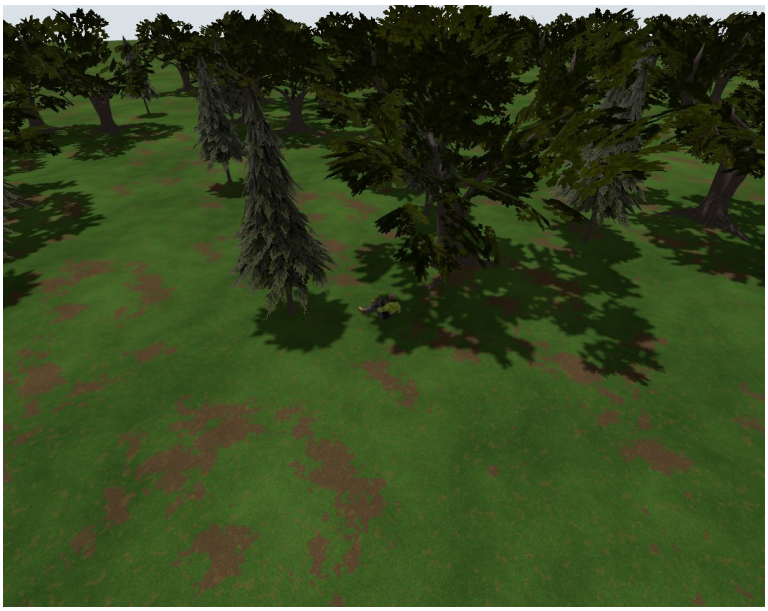


Figura 14.2: Una persona sentada bajo los árboles: desde arriba, una copa puede taparla por completo.

Capítulo 15

Un bosque hecho por factores

En este capítulo

Cómo se genera cada bosque a partir de cuatro factores (oclusión, iluminación, viento y semilla), cómo se mide la cobertura de copas y dónde se colocan las personas.

15.1. Por qué un bosque "por factores"

Para saber si una estrategia de búsqueda es mejor que otra hay que compararlas en condiciones controladas: el mismo bosque, las mismas personas, cambiando una cosa a la vez. Por eso cada bosque se genera a partir de **factores**:

Oclusión (baja, media, alta): fracción del suelo cubierta por copas, 15, 45 y 75 %.

Iluminación (alta, baja): sol de mediodía o sombras largas de tarde.

Viento (no, sí): 4 m/s medios con ráfagas (`WindEffects`).

Semilla: un número que fija dónde quedan los árboles y las personas; con la misma semilla sale exactamente el mismo bosque.

```
python3 scripts/generar_mundo.py --oclusion media --iluminacion alta --viento no --semilla 1 --
    salida ~/mundos
```

15.2. Cobertura de copas

Los robles miden 10 m de diámetro de copa y los pinos 3.2 m. El generador coloca árboles al azar y, sobre una rejilla de 0.5 m, mide qué fracción del cuadrado de 120×120 m queda bajo alguna copa; se detiene al llegar a la cobertura pedida. Deja libres 8 m alrededor del despegue.

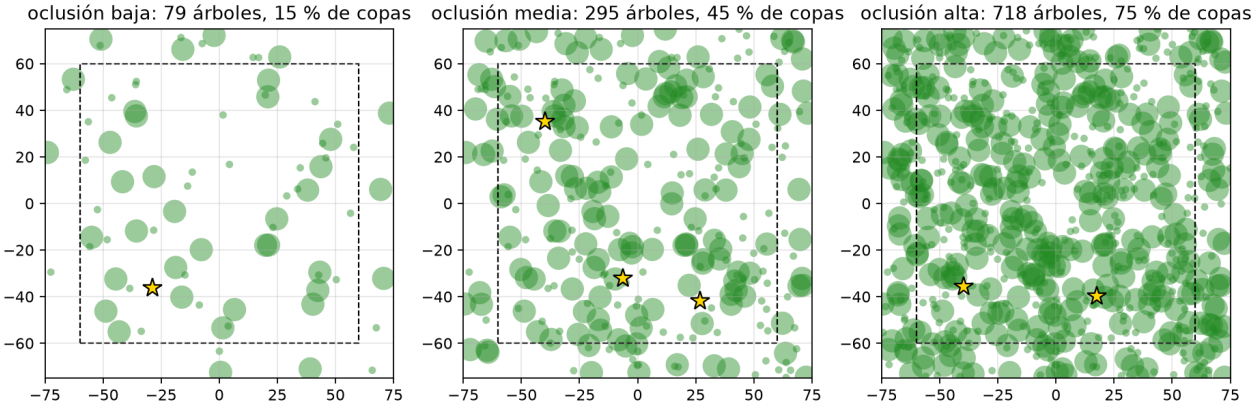


Figura 15.1: Los tres niveles de oclusión con la misma semilla: 79, 295 y 718 árboles. Las estrellas son las personas.

Nota

Las densidades fijas del diseño original (60, 160 y 280 árboles por hectárea) suponían copas de 6 m; con los robles de Gazebo Fuel (10 m) daban otra cobertura. Por eso aquí el nivel se define por la cobertura medida, que es lo que de verdad importa para la cámara, y la densidad resultante se guarda en el JSON.

15.3. Las personas

De una a tres personas (o las que se pidan con `personas:=`), en posiciones al azar a más de 10 m del borde, en tres poses: de pie, sentada o acostada. Sus posiciones exactas se guardan en `personas_<mundo>.json`: la "verdad" contra la que se miden los resultados, que la misión nunca usa para decidir.

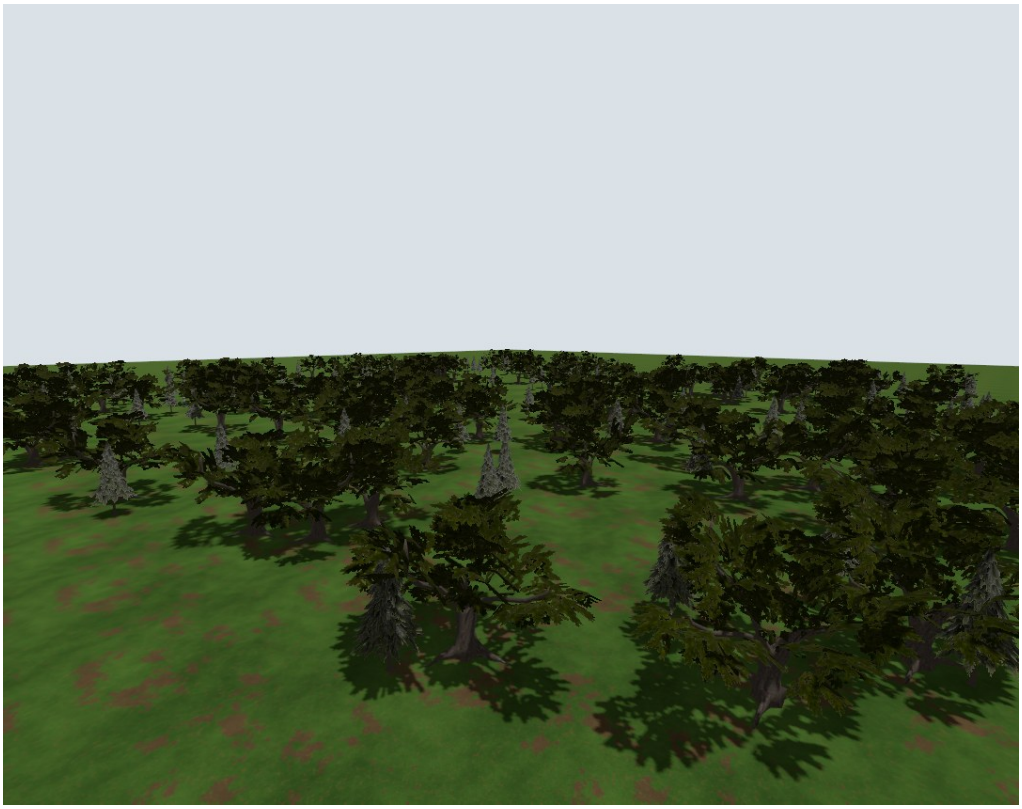


Figura 15.2: El bosque de oclusión media visto a baja altura.

15.4. Georreferencia

Cada mundo declara su ubicación en la Tierra:

```
<spherical_coordinates>
  <surface_model>EARTH_WGS84</surface_model>
  <world_frame_orientation>ENU</world_frame_orientation>
  <latitude_deg>19.4826</latitude_deg>
  <longitude_deg>-99.2447</longitude_deg>
  <elevation>2250</elevation>
</spherical_coordinates>
```

Con eso el GPS simulado de PX4 da coordenadas reales, el *home* de ArduPilot se pone en el mismo punto, y los avisos de personas (que usan la misma conversión, `mision/geo.py`) coinciden con lo que muestra el mapa de MAVProxy.

Parte V

La misión: buscar sin piloto

Capítulo 16

Conceptos de autonomía

En este capítulo

El ciclo percibir–decidir–actuar, las máquinas de estados, cómo se planea recorrer toda una zona, cuándo vale la pena volver a mirar, qué es el GPS y cómo se pasa de metros a latitud y longitud, y qué es un LiDAR.

16.1. Percibir, decidir, actuar

Un dron autónomo repite sin parar tres pasos. *Percibir*: la cámara, el detector y el localizador le dicen dónde podría haber personas. *Decidir*: la misión elige a dónde ir (seguir la franja, volver a mirar una candidata dudosa, aterrizar). *Actuar*: manda la consigna a PX4 o ArduPilot, que mueven los motores.

16.2. Máquinas de estados

La misión se organiza en **estados**: despegue, barrido, re-observación, aterrizaje y fin. En cada uno hace una cosa y cambia de estado cuando se cumple una condición ("llegué a 30 m", "terminé la última franja"). Es fácil de entender, de probar y de registrar: el JSON de cada vuelo trae la lista de cambios de estado con su hora.

Ejemplo de la vida diaria

Una lavadora: llenar, lavar, enjuagar, centrifugar. Cada etapa termina por tiempo o por un sensor, y pasa a la siguiente.

16.3. Cubrir toda la zona

Si no se sabe dónde está la persona hay que mirar todo: **planificación de cobertura**. Lo clásico es ir y venir en franjas paralelas, como al podar un jardín. La clave es la separación: con la cámara hacia abajo a 30 m, la huella mide 71×53 m; con franjas separadas 42 m (60 % de la huella) cada

punto se ve al menos dos veces y queda margen para el viento.

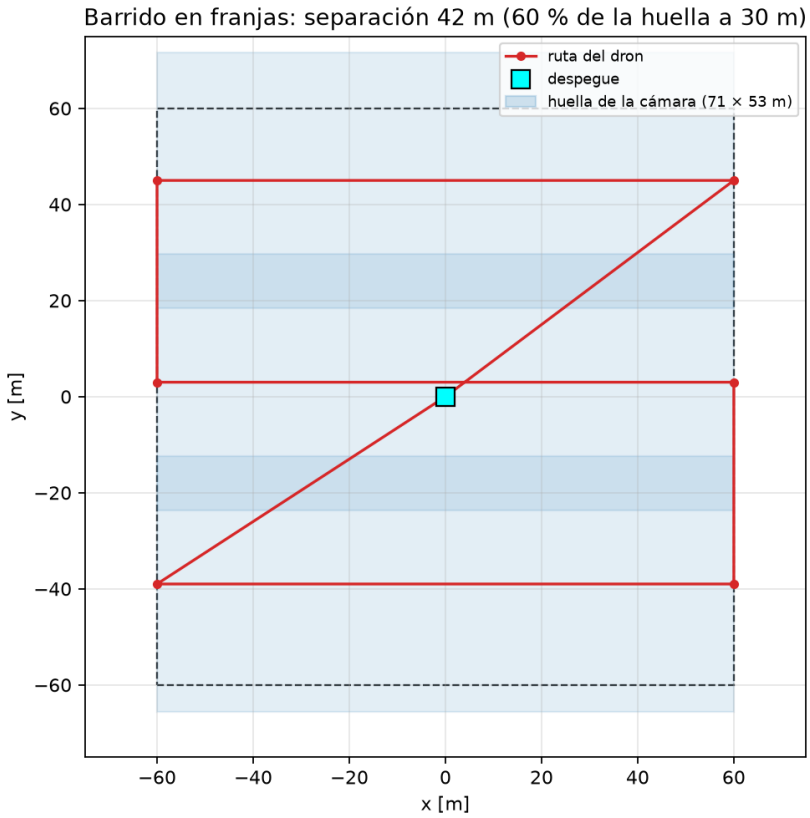


Figura 16.1: El barrido del proyecto: tres franjas cubren el cuadrado de 120 m con traslape.

16.4. Volver a mirar: el valor de la información

Una candidata con confianza intermedia (por ejemplo entre 0.3 y 0.6) es dudosa: puede ser una persona tapada o una sombra. Bajar a verla desde otro ángulo cuesta tiempo y batería, pero puede convertir una duda en una persona encontrada (o descartar una falsa alarma). Esa es la estrategia de **re-observación**: ¿compensa? Responder esa pregunta con datos es el objetivo de la campaña experimental (capítulo 19).

16.5. GPS: de metros a latitud y longitud

La simulación trabaja en metros sobre un plano (ENU); los rescatistas necesitan **coordenadas GPS**. La Tierra se modela como un elipsoide (**WGS84**). La conversión exacta pasa el punto del plano local a coordenadas cartesianas centradas en la Tierra (ECEF) y de ahí a latitud, longitud y altitud (`mision/geo.py`).

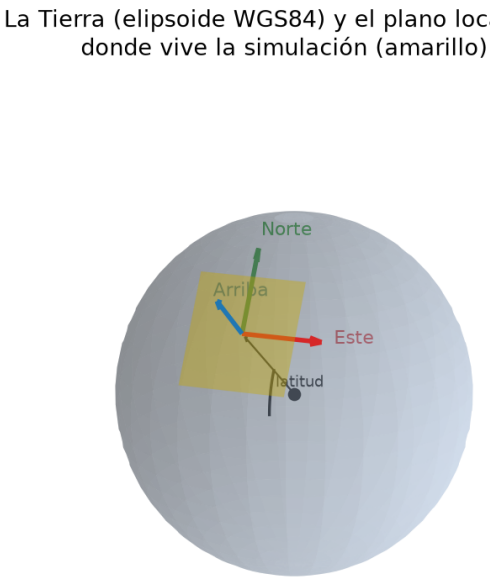


Figura 16.2: El plano local ENU de la simulación es tangente a la Tierra en el origen del mundo.

Para curiosos (te lo puedes saltar)

A 19.5° de latitud, un grado de latitud mide unos 110.8 km y uno de longitud unos 105 km: seis decimales en las coordenadas equivalen a ≈10 cm.

16.6. LiDAR

Un **LiDAR** mide la distancia al primer obstáculo en muchas direcciones lanzando pulsos láser. Da una nube de puntos: no sabe *qué* es cada punto, pero sí dónde está. La cámara reconoce personas; el LiDAR evita chocar.

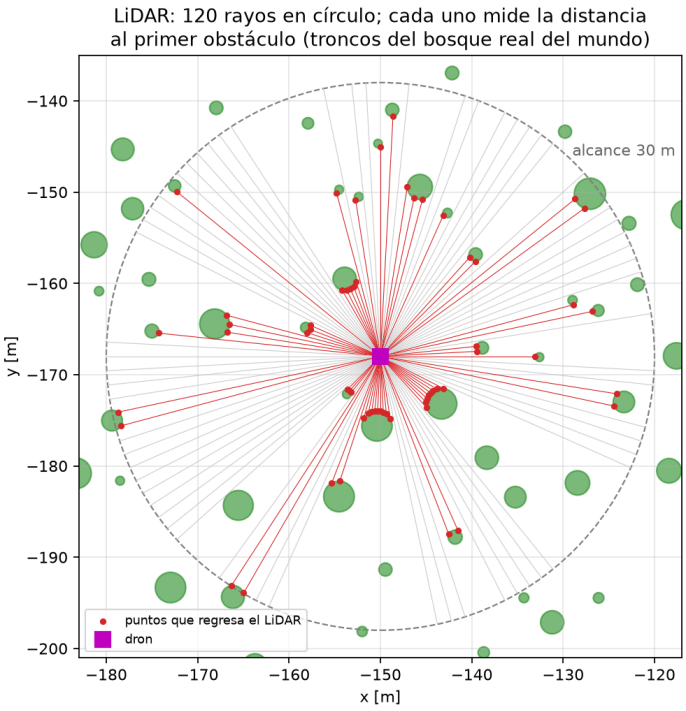


Figura 16.3: Una capa de un LiDAR en un bosque: cada rayo rojo toca un tronco.

La evasión del proyecto es reactiva, del tipo VFH: divide las direcciones alrededor del dron en 72 sectores, marca los bloqueados por obstáculos cercanos y toma el sector libre más parecido a la dirección deseada.

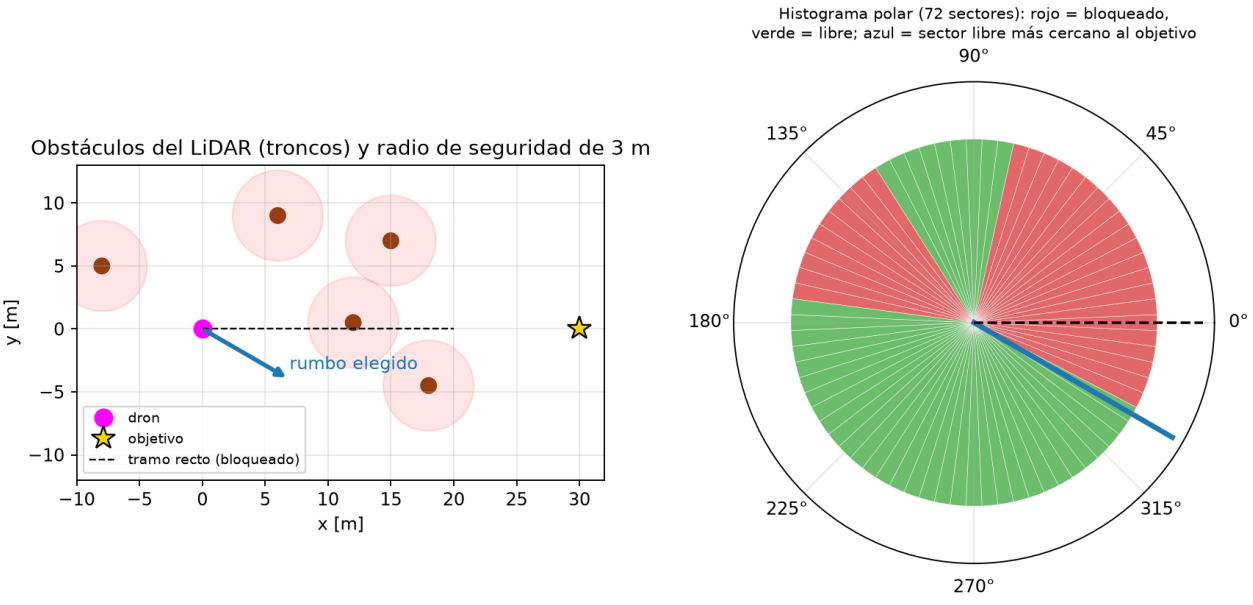


Figura 16.4: Evasión con histograma polar.

Capítulo 17

La misión de búsqueda, pieza por pieza

En este capítulo

Cómo está programada la misión (`mision_node`): el barrido, la re-observación de candidatas dudosas (Algoritmo 1), el cambio a la red neuronal con la consigna adelantada, los avisos GPS, la evasión y las métricas que se guardan al final.

17.1. El ciclo

La misión corre a 20 Hz. En cada ciclo publica una consigna de posición (`TrajectorySetpoint` en NED) y el modo *offboard*; tras el primer segundo pide a PX4 cambiar a offboard y armar.

Despegue: sube a la vertical del origen hasta la altura de trabajo (30 m).

Barrido: recorre las franjas. La consigna avanza a 3 m/s sin alejarse más de 3 m del vehículo (`adelanto_max`). Pasa al siguiente punto de la ruta cuando el vehículo está a menos de 1.5 m del punto y la consigna ya llegó a él.

Re-observación (`estrategia:=reobs`): al terminar cada franja, si hay candidatas dudosas (confianza entre $s_1 = 0.3$ y $s_2 = 0.6$), va a la más cercana: baja a 20 m, con un desplazamiento lateral de 6 m para verla desde otro ángulo, espera 3 s y regresa. Cada candidata se re-observa a lo más dos veces (Algoritmo 1).

Aterrizaje: al terminar, devuelve el control al piloto (si volaba la red) y aterriza.

Fin: calcula las métricas, escribe el JSON y termina; el *launch* se cierra solo.

17.2. Con la red neuronal

Con `controlador:=red`, al llegar a la altura de trabajo la misión cambia al modo *Neural Control* (el identificador lo da PX4 al registrar el modo) y regresa a offboard antes de aterrizar. Además manda la **consigna adelantada** $p_{ref} + 1.6\, v_{ref}$ (1.25 en vertical), que convierte a la red —que imita una PD— en una PD con prealimentación (capítulo 11). El registro guarda la consigna sin adelanto.

Problema encontrado y cómo se resolvió

Se probó suavizar el adelanto con un filtro de 1 s para que en las esquinas la consigna no saltara 4.8 m de lado. Empeoró: el adelanto filtrado seguía apuntando en la dirección vieja y el dron se pasaba de la esquina (RMSE de la ruta de 0.23 a 0.37 m, con el x500). Sin filtro es exactamente una PD con prealimentación; el filtro quedó como opción (`tau_adelanto`) desactivada.

17.3. Avisos con GPS

Cuando una candidata se confirma durante el barrido —confianza acumulada ≥ 0.6, al menos 10 detecciones fusionadas, dentro del área y a más de 5 m de las ya avisadas— la misión publica una sola vez:

- `/personas/gps`: `NavSatFix` con latitud, longitud y altitud (`frame_id` `persona_k`);
- `/personas/alerta`: el texto del aviso;

ambos con QoS *transient local* (quien se suscriba tarde también los recibe). Además lo escribe en la terminal y en `~/personas_localizadas.csv`:

```
PERSONA 2 LOCALIZADA 2026-10-23 07:50:56
  GPS: 19.482212, -99.245049  (19°28'55.9"N 99°14'42.2"W)
  Mapa: https://www.google.com/maps?q=19.482212,-99.245049
  Local: (-40.0, 35.2) m   confianza 1.00, 10 detecciones
```

y en el JSON del mapa que dibuja MAVProxy.



Figura 17.1: El mapa de MAVProxy durante una misión: la persona 1 marcada con sus coordenadas.

Problema encontrado y cómo se resolvió

La primera versión avisó 8 veces para 3 personas: una vista durante el despegue (con 5 m de error), dos a 250 m (rayos casi horizontales en las vueltas) y otras en grupos de robles. Se agregaron tres filtros: solo durante el barrido, solo dentro del área y el límite de 60° en el localizador. Quedaron 4 a 6 avisos para 3 personas: las falsas alarmas persistentes de los robles alcanzan las mismas 10 detecciones que las personas reales, así que subir el umbral no las separa. Resolverlas toca al detector o a la re-observación.

17.4. Evasión de obstáculos

Los puntos del LiDAR pasan al marco del mundo con la pose estimada por el piloto (convertida de NED-FRD a ENU-FLU), se descartan los del suelo y se guardan los cercanos. Si el tramo recto hacia el siguiente punto pasa a menos de 3 m de un obstáculo a la altura del dron, la consigna toma el rumbo libre más cercano o sube (`mision/evasion.py`, el mismo algoritmo del capítulo anterior). A 30 m sobre copas de 6.5 m no actúa durante el barrido: protege los tramos bajos y cualquier cambio de altura o de bosque.

17.5. Métricas

Al terminar, la misión compara las candidatas con la verdad (`metricas.py`) y guarda en el JSON:

`personas_localizadas`	personas con una candidata detectada a menos de 5 m (asignación uno a uno)
`falsas_alarmas`	candidatas detectadas que no corresponden a nadie
`error_localizacion_m`	distancia media entre persona y candidata
`rmse_seguimiento`	distancia RMS del dron a la ruta durante el barrido (sin los desvíos)
`reobservaciones`, `duracion_s`	desvíos hechos y duración
`por_min_detecciones`	lo mismo exigiendo 1, 3, 5 o 10 detecciones fusionadas
`avisos`, `trayectoria`, `eventos`	avisos GPS, trayectoria a 25 Hz y cambios de estado

Capítulo 18

Volar con PX4 o con ArduPilot

En este capítulo

Cómo se conecta la misión a cada piloto automático, cómo corre la red neuronal en cada uno, y la estación de tierra (MAVProxy) con sus ventanas y su mapa.

18.1. Con PX4 (`piloto:=px4`, por omisión)

El *launch* abre Gazebo con el bosque (sin la configuración de PX4), arranca el agente uXRCE-DDS en el puerto 8888 y PX4 SITL (`px4_sitl_neural`, *airframe* 4100, `4100_gz_tarot650_busqueda`; con `vehiculo:=x500`, el 4099) conectado a ese Gazebo (`PX4_GZ_STANDALONE`). La misión habla con PX4 por ROS 2: lee `/fmu/out/vehicle_odometry` y `/fmu/out/vehicle_status_v1`, y publica `/fmu/in/trajectory_setpoint`, `/fmu/in/offboard_control_mode` y comandos (`/fmu/in/vehicle_command`). La red corre dentro de PX4, en `mc_nn_control`.

18.2. Con ArduPilot (`piloto:=ardupilot`)

El mismo bosque, cámara, detector, localizador y lógica de misión, pilotados por ArduCopter 4.7.2 SITL:

- El vehículo es `tarot650_busqueda_ardupilot` (generado por `scripts/generar_vehiculo.py`): el mismo Tarot con `ArduPilotPlugin` (salidas de 1000–2000 µs → 0–680 rad/s, mismo orden de motores que PX4), una IMU para ArduPilot, `OdometryPublisher` a 400 Hz y el sistema `SelectorMotores` (paquete `mision_gz_plugins`). El mundo corre a 1 kHz e incluye el vehículo.
- ArduCopter 4.7.2 se lanza directo (sin MAVProxy ni ventana propia) con los parámetros de `config/ardupilot/tarot650.parm` (los parámetros de vuelo del Tarot: lazos de tasa más suaves que los del Iris de ArduPilot, porque sus motores son más lentos) y el *home* en el origen del mundo.
- `mision_ardupilot` (una subclase de la misión) se conecta por TCP 5760 con MAVLink, vuela en GUIDED con `SET_POSITION_TARGET_LOCAL_NED` y publica el estimado del EKF de ArduPilot en `/fmu/out/vehicle_odometry`, para que el localizador sea exactamente el mismo.

- Con `controlador:=red`, la red corre en `controlador_red` (ROS 2 + gz-transport, ∼330 Hz, 70 µs por paso) con el estado de Gazebo y manda $\omega_i = \omega_{max}\sqrt{(a_i+1)/2}$ rad/s (ω_{max} = 680 rad/s en el Tarot). La misión publica "red" en `/dron/controlador` y `SelectorMotores` deja pasar esos comandos en lugar de los de ArduPilot, que sigue estimando, recibe las mismas consignas y recupera los motores para aterrizar, o si la red deja de publicar 50 ms.

18.2.1. ArduPilot vuela con su EKF3

ArduCopter vuela con su propio estimador, el **EKF3** (`AHRS_EKF_TYPE3` en `tarot650.parm`): calcula la posición y la orientación a partir de la IMU, el GPS, el barómetro y el magnetómetro simulados, igual que en un dron real. Para eso el modelo lleva `no_time_sync=0`. Con `no_time_sync=1` (lo que traen los modelos de `ardupilot_gazebo`) el enlace JSON entre Gazebo y ArduPilot fuerza `AHRS_EKF_TYPE=10` y ArduPilot vuela con la posición y la actitud *verdaderas* del simulador. La IMU de ArduPilot tiene ruido y sesgo supuestos (giróscopo 0.004 rad/s, acelerómetro 0.1 m/s^2): sin ruido, el EKF tendría una tarea mucho más fácil que en un dron real.

Cuidado

Las corridas con ArduPilot anteriores al 24 de octubre usaban el estado verdadero del simulador, no el EKF: el paquete tenía `no_time_sync=1`. Tenlo en cuenta al compararlas con las nuevas o con las de PX4, que vuela con su propio estimador (EKF2).

Para conectar una estación de tierra a ArduPilot: `mavproxy.py--mastertcp:127.0.0.1:5762`.

18.3. La estación de tierra: MAVProxy

Con GUI, el *launch* abre MAVProxy en su propia terminal, con consola, mapa y el módulo `mi sion.mavproxy_trayectoria`, conectado a PX4 por UDP 14550 o a ArduPilot por TCP 5762; `ventanas_sitl` acomoda las ventanas (terminal y consola a la izquierda, Gazebo al centro, mapa a la derecha). El mapa dibuja la ruta planeada, la trayectoria recorrida y cada persona avisada.

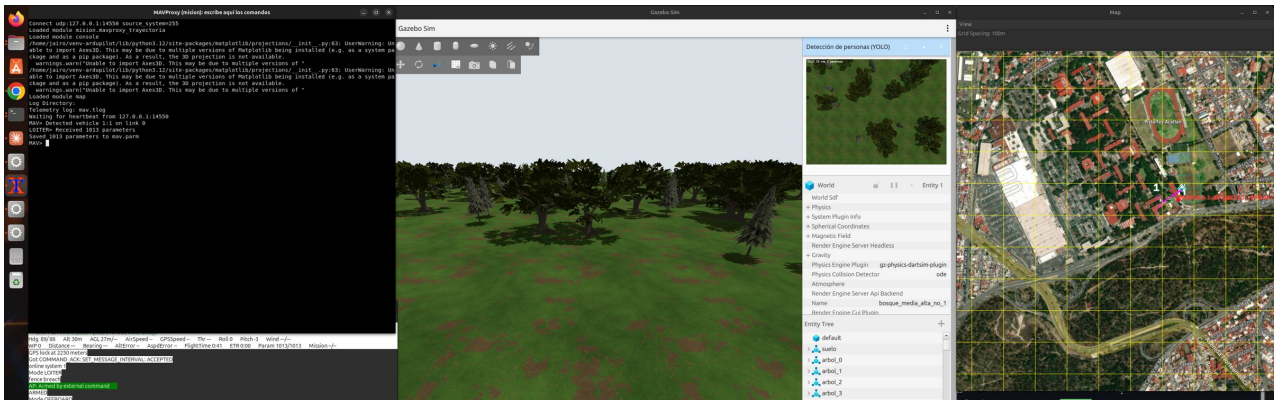


Figura 18.1: MAVProxy conectado a PX4: la terminal (con el prompt para escribir comandos), la consola, Gazebo y el mapa.

Comando en MAVProxy	Efecto
`modeLOITER` / `modeRTL` / `modeLAND`	detenerse, regresar al despegue, aterrizar
`armthrottle` / `disarm`	armar / desarmar
`paramshowMPC_*`	ver parámetros (PX4)
`trayectoriaborrar|ocultar|mostrar`	dibujo de la trayectoria en el mapa

Cuidado

Si tomas el control desde MAVProxy, la misión sigue mandando consignas de offboard (PX4) o GUIDED (ArduPilot). Al regresar a ese modo el dron retoma la búsqueda.

18.4. Una simulación a la vez

El *launch* usa `/clock`, `/camera` y el puerto 8888 del agente. Para correr otra simulación al mismo tiempo (por ejemplo, otra de ROS en la misma computadora), lánzala en otra partición de Gazebo y otro dominio de ROS:

```
GZ_PARTITION=busqueda ROS_DOMAIN_ID=7 ros2 launch mision simulacion.launch.py
```

Dos PX4 al mismo tiempo no se puede sin cambiar puertos: por eso la campaña corre los vuelos uno por uno.

Parte VI

Experimentos y manos a la obra

Capítulo 19

La campaña experimental

En este capítulo

Qué preguntas se quieren responder, cómo se diseñan los vuelos para responderlas sin engañarse, la prueba de Wilcoxon con la corrección de Holm, y lo que dijo la campaña.

19.1. Las preguntas

P1. ¿La re-observación de candidatas dudosas encuentra más personas que el barrido fijo, y en qué nivel de oclusión ayuda más?

P2. ¿Cuánto cuesta? Tiempo de vuelo, personas por minuto y desvío de la ruta.

P3. ¿Qué tan sensible es la misión a un detector más lento o que pierde cuadros (`latencia_extra_ms`, `saltar_cuadros`)?

P4. ¿Cambia algo volar con la red neuronal en vez de la cascada de PX4?

19.2. El diseño

Un **diseño factorial completo**: todas las combinaciones de oclusión (3), iluminación (2), viento (2) y estrategia (2), es decir 24 vuelos por semilla con el controlador en cascada. Para P4 se agregan, en oclusión media y sol alto, los 4 vuelos de la red (con y sin viento, las dos estrategias): 28 vuelos por semilla.

```
python3 src/mision/scripts/correr_experimentos.py --semillas 30 --salida ~/resultados     # 840
    vuelos
python3 src/mision/scripts/correr_experimentos.py --semillas 2 --solo oclusion=media      # una
    parte
```

Cada vuelo es un `ros2launch` sin interfaz en su propia partición de Gazebo y su dominio de ROS 2, dura unos 5 minutos y deja un JSON. El programa salta los vuelos que ya tienen JSON, así que se puede interrumpir y continuar, o ampliar de 5 a 30 semillas.

Consejo

Apareado. La re-observación y el barrido fijo vuelan *el mismo bosque* (misma semilla, iluminación y viento). La diferencia entre los dos vuelos de un par se debe a la estrategia y no a que a uno le tocó un bosque más difícil. Así se necesitan muchos menos vuelos para ver un efecto.

19.3. Wilcoxon y Holm

Para cada par se calcula la diferencia $d = \text{reobs} - \text{fija}$ (personas encontradas, por ejemplo). La **prueba de rangos con signo de Wilcoxon** pregunta si esas diferencias tienden a ser positivas, sin suponer que siguen una campana de Gauss: ordena las $|d|$, les pone rango y compara la suma de los rangos positivos con la de los negativos. Se reporta la mediana de d con un intervalo de confianza del 95 % obtenido por remuestreo (*bootstrap*).

Se hacen 12 pruebas (3 oclusiones $\times$ 4 variables). Con tantas, alguna saldría "significativa" por pura suerte; la **corrección de Holm** lo evita: ordena los valores p de menor a mayor y multiplica el k-ésimo por $(m - k + 1)$, sin dejar que bajen.

```
~/venv-mision/bin/python src/mision/scripts/analisis_wilcoxon.py --resultados ~/resultados
~/venv-mision/bin/python src/mision/scripts/analisis_wilcoxon.py --demo        # datos sintéticos
```

Cuidado

Una sola semilla no compara estrategias: en el primer bosque la re-observación puede ganar o perder por casualidad. Las conclusiones salen de los pares, no de un vuelo bonito.

19.4. Resultados

Las tablas siguientes las genera `herramientas/tabla_campana.py` a partir de los JSON de `~/resultados`; se regeneran al agregar semillas.

Corridas analizadas: 8 (semillas 1–1).

Control	Oclusión	Estrategia	n	Encontr.	FA	FA_{10}	Avisos	Dur. (s)	RMSE (m)
cascada	baja	fija	4	4/4	16.5	5.8	4.2	251	0.255
cascada	baja	reobs	4	4/4	20.2	8.0	4.5	304	0.342

Tabla 19.1: Promedios por condición. FA: falsas alarmas con una detección; FA_{10}: con al menos 10; avisos GPS por vuelo.

Oclusión	Variable	n	Mediana d	IC 95 %	p	p_{Holm}
baja	personas	4	0.000	[0.000, 0.000]	1.000	1.000
baja	personas/min	4	-0.040	[-0.048, -0.034]	1.000	1.000
baja	RMSE (m)	4	0.083	[-0.029, 0.210]	0.625	1.000
baja	falsas alarmas	4	4.500	[-2.000, 8.000]	0.250	1.000
media	personas	0	–	[–, –]	–	–
media	personas/min	0	–	[–, –]	–	–
media	RMSE (m)	0	–	[–, –]	–	–
media	falsas alarmas	0	–	[–, –]	–	–
alta	personas	0	–	[–, –]	–	–
alta	personas/min	0	–	[–, –]	–	–
alta	RMSE (m)	0	–	[–, –]	–	–
alta	falsas alarmas	0	–	[–, –]	–	–

Tabla 19.2: Re-observación menos barrido fijo, apareado por semilla, iluminación y viento (controlador en cascada).

Problema encontrado y cómo se resolvió

Las falsas alarmas dependen del umbral. Con una sola detección cualquier sombra que el detector confunda una vez cuenta como candidata (10 a 20 por vuelo); exigiendo 10 detecciones fusionadas quedan pocas, y casi todas son grupos de robles que el detector confunde una y otra vez. Por eso la tabla trae las dos cifras, y por eso el aviso GPS exige 10.

Capítulo 20

Tu primer vuelo, paso a paso

En este capítulo

Lanzar una misión completa, saber qué mirar en cada ventana, leer el resultado y cambiar una cosa a la vez.

20.1. Antes de empezar

- La instalación del capítulo 12 ya está hecha y compilada (`colconbuild`).
- No hay otra simulación de PX4 abierta (comparten el puerto 8888).
- Si la campaña está corriendo, espera a que termine o usa otra partición y otro dominio (capítulo 18).

20.2. Paso 1: lanzar

```
source ~/ros2_ws/install/setup.bash
```

```
ros2 launch mision simulacion.launch.py
```

En unos 30 segundos aparecen Gazebo con el bosque, la terminal y la consola de MAVProxy, y el mapa. El Tarot 650 despega del origen. La terminal de ROS va diciendo lo que pasa (esta salida es de una corrida del 23 de octubre, con el x500; con el Tarot los mensajes son los mismos):

```
[mision]: misión fija con controlador cascada: 8 puntos de ruta a 30.0 m
[mision]: despegue -> barrido
PERSONA 1 LOCALIZADA 2026-10-23 08:28:21
  GPS: 19.482273, -99.244990  (19°28'56.2"N 99°14'42.0"W)
[mision]: barrido -> aterrizaje
[mision]: 1/1 personas localizadas, 14 falsas alarmas, error 0.43
[mision]: misión terminada en 237 s; RMSE de seguimiento 0.07 m; registro en ...
```

20.3. Paso 2: mirar

Gazebo: el dron, el bosque y, a la derecha, el panel "Detección de personas" con la imagen de la cámara y los recuadros de YOLO.

Mapa de MAVProxy: la ruta en franjas, la trayectoria recorrida y cada persona avisada.

Terminal: los avisos con GPS y enlace a Google Maps.

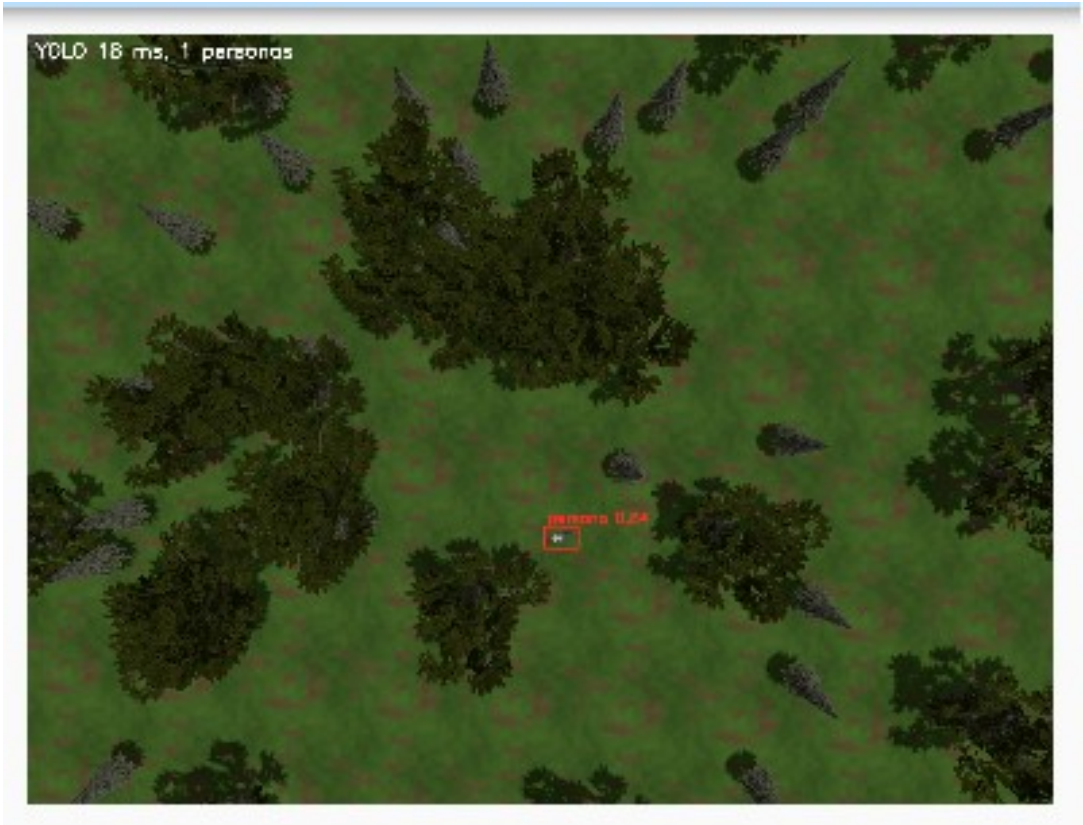


Figura 20.1: El panel de detección durante el barrido.

20.4. Paso 3: leer el resultado

```
python3 -c "import json; r=json.load(open('$HOME/resultados/corrida.json')); \
print(r['personas_localizadas'], '/', r['personas'], r['falsas_alarmas'], r['duracion_s'])"
```

El significado de cada campo está en la tabla de métricas del capítulo 17.

20.5. Paso 4: cambiar una cosa

```
ros2 launch mision simulacion.launch.py estrategia:=reobs            # con re-observación
ros2 launch mision simulacion.launch.py oclusion:=alta semilla:=4    # bosque denso
```

```
ros2 launch mision simulacion.launch.py controlador:=red              # vuela la red neuronal
ros2 launch mision simulacion.launch.py piloto:=ardupilot             # con ArduPilot
ros2 launch mision simulacion.launch.py vehiculo:=x500                # con el x500 de PX4
ros2 launch mision simulacion.launch.py gui:=false salida:=$HOME/r.json   # sin ventanas
```

Pruébalo tú

Vuela la semilla 1 en oclusión alta con `estrategia:=fija` y luego con `reobs`. ¿Cuántas personas encontró cada una, y cuánto tardó? Luego repite con la semilla 2. ¿Se mantiene la conclusión?

Pruébalo tú

Vuela con `latencia_extra_ms:=300`. ¿Cuánto crece el error de localización? ¿Por qué? (Pista: a 3 m/s, 300 ms son 0.9 m.)

20.6. Si algo falla

Síntoma	Causa y remedio
El dron no despega	PX4 no recibe la odometría o el agente no arrancó: revisa que no haya otra simulación en el puerto 8888 (`pgrep -a MicroXRCE`).
La memoria se llena en minutos	Gazebo cargó `GstCameraSystem`; el *launch* lo evita, no uses `make px4_sitl gz_...` para estas misiones.
Ninguna persona encontrada	¿El detector cargó los pesos? Busca `[detector]` en la terminal y mira `/detecciones`.
Quedan ventanas abiertas	`pkill -f mavproxy`; el *launch* normalmente las cierra.
Con `vehiculo:=x500` y `controlador:=red` PX4 vuela mal	PX4 lleva compilada la red del Tarot; reinstala la del x500 con `instalar_px4.sh` (capítulo 14).

Capítulo 21

Referencia rápida

En este capítulo

Los argumentos del *launch*, los parámetros de la misión, los tópicos y los programas, en tablas para consultar.

21.1. Argumentos de `simulacion.launch.py`

Argumento	Por omisión	Qué hace
`vehiculo`	tarot650	tarot650 o x500 (el de las primeras corridas)
`oclusion`	media	baja, media, alta: cobertura de copas 15, 45, 75 %
`iluminacion`	alta	alta (mediodía) o baja (sombras largas)
`viento`	no	si: 4 m/s medios con ráfagas
`semilla`, `personas`	1, (1 a 3)	bosque y número de personas
`estrategia`	fija	fija o reobs (Algoritmo 1)
`piloto`	px4	px4 o ardupilot
`controlador`	cascada	cascada (del piloto) o red
`altura`, `velocidad`, `lado`	30, 3, 120	m, m/s, m
`pesos`, `umbral`	YOLO26s, 0.25	modelo del detector y confianza mínima
`pesos_red`	`pesos_nn_tarot650_busqueda.npz`	pesos de la red para `controlador_red` (ArduPilot); por omisión, los del vehículo
`latencia_extra_ms`, `saltar_cuadros`	0, 0	degradación artificial del detector (P3)
`gui`, `mavproxy`	true, true	ventanas de Gazebo y estación de tierra
`salida`	`~/resultados/corrida.json`	registro del vuelo
`px4`, `ardupilot`	`~/PX4-Autopilot`, `~/ardupilot`	dónde están los pilotos

21.2. Parámetros de `mision_node`

Parámetro	Por omisión	Qué hace
`adelanto_max`	3.0 m	distancia máxima de la consigna por delante del dron
`huella`	70 m	ancho de la huella para planear franjas
`s1`, `s2`	0.3, 0.6	confianza de una candidata dudosa
`max_reobs`, `altura_reobs`, `espera_reobs`	2, 20 m, 3 s	re-observación
`radio_acierto`, `min_detecciones`	5 m, 1	para las métricas
`tiempo_max`	900 s	corta la misión si se pasa
`prealimentacion`	-1 (auto)	consigna adelantada con la red
`tau_adelanto`	0 s	filtro del adelanto (desactivado)
`n_aviso`	10	detecciones para avisar con GPS
`registro_personas`	`~/personas_localizadas.csv`	bitácora de avisos
`origen_gps`	19.4826, -99.2447, 2250	origen del mundo
`evasion`, `radio_evasion`, `alcance_evasion`	true, 3 m, 20 m	evasión con LiDAR

21.3. Tópicos

Tópico	Tipo	Quién lo publica
`/camera`, `/camera_info`	Image, CameraInfo	Gazebo (puente)
`/detecciones`	Detection2DArray	detector
`/camera_yolo`	Image	detector (imagen con recuadros)
`/detector/latencia_ms`	Float32	detector
`/personas/estimaciones`	Detection3DArray	localizador (cada detección)
`/personas/candidatas`	Detection3DArray	localizador (fusionadas)
`/personas/gps`, `/personas/alerta`	NavSatFix, String	misión
`/dron/consigna`	PointStamped	misión
`/dron/lidar/points`	PointCloud2	Gazebo (puente)
`/fmu/out/vehicle_odometry`	VehicleOdometry	PX4 (o `mision_ardupilot`)
`/fmu/in/trajectory_setpoint`	TrajectorySetpoint	misión

21.4. Programas

Programa	**Para qué**
`scripts/generar_mundo.py`	bosque por factores
`scripts/generar_vehiculo.py`	modelos `tarot650_busqueda` y `x500_busqueda` (PX4 y ArduPilot)
`scripts/importar_personas.py`	modelos de personas desde Gazebo Fuel
`scripts/capturar_dataset.py`, `preparar_datos.py`, `entrenar_detector.py`	datos y entrenamiento del detector
`scripts/correr_experimentos.py`, `analisis_wilcoxon.py`	campaña y estadística
`scripts/instalar_px4.sh`	copia los vehículos, los *airframes* y la red a PX4
`codigo/nn_imitacion.py`, `ppo_refinamiento.py`, `exportar_tflite.py`	entrenamiento y exportación de la red
`mision/prueba_neural.py`	escalones con la red en SITL

Capítulo 22

Lo que se verificó y lo que falta

En este capítulo

Los problemas que aparecieron al llevar el diseño a la computadora, cómo se resolvieron, y lo que queda pendiente.

22.1. Diferencias con el diseño original

El diseño original del que parte el proyecto tenía nueve detalles que no funcionaban tal cual. Todos están corregidos en el paquete:

1. La etiqueta `v1.17.0` de PX4 no existe; se usa la rama `release/1.17`.
2. El sensor de la cámara se llama `imager`, no `camera`.
3. `VehicleStatus` es versionado: el tópico es `/fmu/out/vehicle_status_v1`.
4. En `x500_mono_cam_down` la cámara está 10 cm *arriba* del centro y ve el propio vehículo; aquí va 25 cm abajo en el x500 y 24 cm abajo en el Tarot 650.
5. La configuración de Gazebo de PX4 carga `GstCameraSystem`, que acumula cuadros si el codificador se atora: 110 MB/s y el sistema se queda sin memoria en minutos. Aquí Gazebo arranca sin él.
6. `MC_NN_THRST_COEF` trabaja con revoluciones por segundo, no rpm, y faltaban `MC_NN_MIN_RPM` y `THR_MDL_FAC`; los valores correctos están en los *airframes* 4100 (Tarot 650) y 4099 (x500) y los verifican las pruebas.
7. La masa del x500 completo es 2.114 kg, no 2.0.
8. `exportar_tflite.py` usaba `--salida` como nombre del símbolo en C; el módulo exige `control_net_tflite`.
9. PX4 marca sus mensajes con el reloj de pared, no con el de Gazebo; el localizador estima el desfase con el mínimo de (llegada − marca).

22.2. Lo que se comprobó

- Los programas de modelo, localización, imitación, PPO y exportación reproducen las tablas y salidas de referencia (el TFLite difiere de numpy en 1.1×10^{-6}).
- El detector ajustado: AP50 0.954 en validación; el YOLO de COCO no encontraba a nadie.
- La red mejorada (con el x500): sesgo estacionario de 25.9 a 2.1 cm y misión en 211 s en vez de 427 s.
- Las coordenadas GPS de los avisos coinciden con el GPS de PX4, el *home* de ArduPilot y el mapa de MAVProxy (prueba `test_origen_gps_igual_al_del_mundo`).
- `colcontest`: las 8 pruebas del paquete pasan.

22.3. Pendientes

- **Los parámetros del Tarot 650.** Masa, inercias, k_f y retardos de los motores son supuestos. Hay que medirlos en el dron real, regenerar el vehículo, reentrenar la red e instalarla (capítulo 14).
- **Las esquinas con la red.** En los tramos rectos la red sigue la ruta igual que la cascada, pero en las vueltas de 90° la consigna adelantada cambia de dirección de golpe y el dron se pasa (RMSE de 0.27–0.29 m contra 0.08–0.10 m de la cascada). Suavizar la consigna en las esquinas es el siguiente paso.
- **Falsas alarmas persistentes.** Grupos de robles que el detector confunde en muchos cuadros. Reentrenar con esos "negativos difíciles" es el siguiente paso.
- **PPO.** Ya no diverge, pero no mejora a la imitación; falta una recompensa que premie lo que la imitación no sabe hacer (por ejemplo, rechazar viento).
- **Seguimiento de cámara en la GUI.** `/gui/follow` no funciona con esta interfaz.
- **Campaña completa.** Con 5 semillas se ven tendencias; las 30 semillas del diseño dan la potencia estadística para P1.

Apéndice A

Glosario

AP50, AP50-95
Precisión promedio de un detector, contando como acierto un recuadro que se traslapa al menos 50 % (o promediando de 50 a 95 %) con el verdadero.

ArduPilot / ArduCopter
Piloto automático de código abierto; ArduCopter es su versión para multirrotores.

Armar
Habilitar los motores para que puedan girar.

Barrido
Recorrer una zona en franjas paralelas para verla toda.

Bootstrap
Remuestrear los datos muchas veces para estimar la incertidumbre de una cifra.

Candidata
Posible persona en el mapa: varias detecciones fusionadas en un punto.

Cascada
Lazos de control anidados: posición afuera, velocidad, actitud y velocidad angular adentro.

Consigna
Lo que se pide al controlador (posición, velocidad).

Consigna adelantada
Consigna desplazada en la dirección del movimiento para que un PD siga una referencia móvil sin atraso.

Cobertura de copas
Fracción del suelo tapada por árboles vistos desde arriba.

Detector
Red que encuentra objetos en una imagen y los marca con recuadros.

Diseño factorial
Experimento que prueba todas las combinaciones de los factores.

EKF
Filtro de Kalman extendido: estima posición y orientación combinando sensores.

ENU / NED
Este-Norte-Arriba (ROS, Gazebo) y Norte-Este-Abajo (PX4, aviación).

Falsa alarma
Candidata que no corresponde a ninguna persona.

FLU / FRD
Marcos del cuerpo: Frente-Izquierda-Arriba (ROS) y Frente-Derecha-Abajo (PX4).

Gazebo
Simulador de robots con física, render y sensores.

GUIDED
Modo de ArduPilot en el que obedece consignas de una computadora.

Holm
Corrección que ajusta los valores p cuando se hacen muchas pruebas a la vez.

Huella
Rectángulo del suelo que ve la cámara.

Imitación
Entrenar una red para que copie las salidas de otro controlador.

LiDAR
Sensor que mide distancias con pulsos láser en muchas direcciones.

MAVLink
Protocolo de mensajes entre pilotos automáticos y estaciones de tierra.

MAVProxy
Estación de tierra en terminal, con consola y mapa.

Máquina de estados
Programa organizado en etapas con reglas para pasar de una a otra.

Neural Control
Modo de PX4 en el que una red neuronal manda los motores directamente.

Offboard
Modo de PX4 en el que obedece consignas de una computadora.

Oclusión
Que algo tape lo que se quiere ver.

PPO
Algoritmo de aprendizaje por refuerzo que mejora una política con pasos pequeños.

Prealimentación
Sumar al control lo que ya se sabe que se necesita (por ejemplo, la velocidad de la referencia), sin esperar a que haya error.

PX4
Piloto automático de código abierto.

Re-observación
Volver a mirar una candidata dudosa desde más cerca y otro ángulo.

RMSE
Raíz del error cuadrático medio.

ROS 2
Conjunto de herramientas para que los programas de un robot se comuniquen por tópicos.

Semilla
Número que fija los resultados al azar; con la misma semilla sale lo mismo.

SITL
Software in the loop: el piloto automático real corriendo contra un simulador.

TFLite
Formato compacto de redes neuronales para microcontroladores.

Tópico
Canal con nombre por el que los nodos de ROS 2 se mandan mensajes.

uXRCE-DDS
Puente entre PX4 y ROS 2.

VFH
Histograma polar de campo vectorial: evasión que busca la dirección libre más cercana.

Wilcoxon
Prueba estadística para diferencias apareadas que no supone una campana de Gauss.

WGS84
Elipsoide que modela la forma de la Tierra para el GPS.

YOLO
Familia de detectores de objetos que mira la imagen una sola vez.